MAGIQ: A Post-Quantum Multi-Agentic AI Governance System with Provable Security

Sepideh Avizeh
University of Calgary
Calgary, Alberta, Canada

Tushin Mallick
Northeastern University
Boston, MA, USA

Alina Oprea
Northeastern University
Boston, MA, USA

Cristina Nita-Rotaru
Northeastern University
Boston, MA, USA

Reihaneh Safavi -Naini
University of Calgary
Calgary, Alberta, Canada

Abstract

Our computing ecosystem is being transformed by two emerging paradigms, the increased deployment of agentic AI systems and the advancements in quantum computing. With respect to agentic AI systems, one of the most critical problems is creating secure governing architectures that ensure agents follow their owner's communication and interaction policies, and can be held accountable for the messages they exchange with other agents. With respect to quantum computing, existing systems must be retrofitted and new cryptographic-based mechanisms must be designed to ensure long-term security and quantum-resistance. In fact, NIST recommends that standard public-key cryptographic algorithms, including RSA, Diffie–Hellman (DH), and elliptic-curve constructions (ECC), be deprecated starting in 2030 and disallowed after 2035.

In this paper we create MAGIQ, a framework for policy definition and enforcement of multi-agentic AI systems using novel highly efficient quantum-resistant cryptographic protocols with proved security guarantees. MAGIQ (i) allows users to define rich communication and access control policy budgets for agent-to-agent sessions and tasks, with global budgets for one-to-many agent sessions; (ii) enforces such policies using post-quantum cryptographic primitives; (iii) supports session-based enforcement of policies for agent-to-agent and one-to-many agent sessions; and (iv) provides accountability of agents to their users in the form of message attribution. We formally model and prove correctness and security of the system using the Universal Composability (UC) framework. We evaluate the computation and communication overhead of our framework and compare it with the state of the art agentic AI framework SAGA. MAGIQ is the first step towards post-quantum secure solutions for agentic AI systems.

1 Introduction

Our society is being transformed by the emergence of agentic AI systems. Agents are now deployed across domains such as finance, healthcare, customer support, and software engineering, to autonomously plan and execute complex, multi-step tasks with minimal human intervention. One of the most critical problems in agentic AI deployment [48] is creating secure governing architectures that ensure defining unique identification of AI agents, authenticating the agents in their interactions, ensuring agents follow their owner's communication and interaction policies, and can be held accountable for their interactions with other agents.

Previous work on agentic governance has focused on defining requirements for agent identities [14], capability taxonomies [41], introducing attribution [15], authorization with delegation capabilities [50], or indexing [45]. These proposals remain theoretical without implementation, empirical evaluation, or provable guarantees. Protocols like Agent2Agent (A2A) [24] and Model Context Protocol (MCP) [18] promote agent interoperability but neither provides policy enforcement mechanisms or runtime mediation of agent interactions, leaving them vulnerable to documented attacks [39, 56]. The only practical system focused on ensuring agents follow policies created by their users is SAGA [52]. SAGA enforces access control policies based on an access control token generated by the agent allowing access for the agent requesting access, with the help of a centralized `Provider` managing a registry of users and agents. SAGA's policies are for agent-to-agent interactions, and consist of user-defined authorized communication budgets of incoming requests per agent. While SAGA provides formal (ProVerif) proofs for the basic security properties of the communicated messages, it does not provide a security model for policy definition and enforcement. In addition, SAGA protocols use classical public key cryptographic primitives.

Recent advancements in quantum computing coupled with the Shor's [49] and Grover's [25] algorithms raise serious concerns about the use of public key cryptographic primitives in systems and weakening of symmetric-primitives. Draft NIST IR 8547 [40] (Transition to Post-Quantum Cryptography Standards) recommends deprecating quantum-vulnerable algorithms (RSA, ECDSA, EdDSA, DH, ECDH) by 2030 and fully retiring them by 2035. To help with the transition, NIST released several PQC standards—FIPS 203 [42], FIPS 204 [43], and FIPS 205 [44] in August 2024. Two more drafts are under development for a digital signature based on FALCON [22], and a key encapsulation algorithm based on the Hamming Quasi-Cyclic (HQC), both expected to have drafts in 2026.

Given the growing deployment and reliance on agentic AI systems, and rapid development of quantum technologies driven by global investment in the field, it is essential to design post-quantum secure governance systems that remain secure in the presence of an adversary with access to a quantum computer.

One solution to creating a post-quantum secure governance system is to replace SAGA's underlying cryptographic primitives with post-quantum counterparts. However, this approach would incur significant communication and computation overhead and, more importantly, would neither address the limitations in policy definition nor provide formal security guarantees for policy enforcement.

Our work. In this paper we introduce MAGIQ, a framework for policy definition and enforcement in multi-agentic AI systems

using novel highly efficient quantum-resistant cryptographic protocols with proved security guarantees. The goal of MAGIQ is to define and enforce policies that are meaningful with respect to agent-semantics, i.e. tasks they need to solve. While agents can communicate securely over PQ-TLS, these transport level messages are decoupled from task semantics and too low-level to capture agent-level interactions.

We introduce two application-level communication abstractions which we refer to as *A-session* and *C-session*. An *A*-session captures one-to-one communication between two agents, whereas a *C*-session captures one-to-many communication between a coordinating agent and a set of other agents. We define the unit of communication on these sessions as *task-msg*. A *task-msg* consists of a structured message that combines intent, data (payload), context, and agent identity, so the receiving agent can understand, verify, and act on the request. These two types of sessions are used to define and enforce policies.

MAGIQ cryptographically enforces four types of policies for an agent: ① the access control list with respect to agents receiving and initiating contact, ② the (maximum) number of *A-sessions* that an agent can initiate, or respond to, ③ the (maximum) number of *task-msg* an agent can send in an *A-session*, taking the role of initiating or receiving party, and ④ the duration of time an agent can be active for a task. These policies prevent access from unauthorized agents, mitigate attacks from malicious agents, and provide protections against different forms of denial-of-service attacks. In addition, because each *task-msg* is individually counted and cryptographically linked to a one-time signature/token, our policies provide attribution with respect to each interaction between agents.

Cryptographic policy enforcement in an *A*-session relies on two mechanisms. A digitally signed session-authorization token enforces the user-defined policy on authorized contactable agents and the allowed number of *A*-session. Two hash chains, one constructed by the agent that initiates the session, A_I and signed by its owner, and one constructed by the agent that responds to the request, A_R, enforce the agents' respective *task-msg* policies in the *A*-session. Policy enforcement for a *C*-session relies on the policy enforcement of its *A*-session, together with an efficiently computed membership token for the session. To summarize our contributions:

- We propose MAGIQ, a system that (i) allows users to define rich communication and access control policy budgets, (ii) enforces such policies using cryptographic primitives that are post-quantum resistant, and (iii) provides accountability of agents in the form of *task-msg* attribution to their users.
- We provide a UC-based formal model for policy-defined and policy-enforced secure agent communication, and prove that the MAGIQ protocols securely realize it.
- We evaluate the overhead of MAGIQ and show that it is practical when compared to the state-of-the art solution SAGA that uses classical cryptography.

The rest of the paper is organized as follows. Section 2 provides background on governance for agentic AI systems and post-quantum primitives. Section 3 describes the attacker model we assume. Section 4 describes the design of the system and Section 5 its security. Section 6 presents evaluation results. Section 7 presents related work and Section 8 concludes our work.

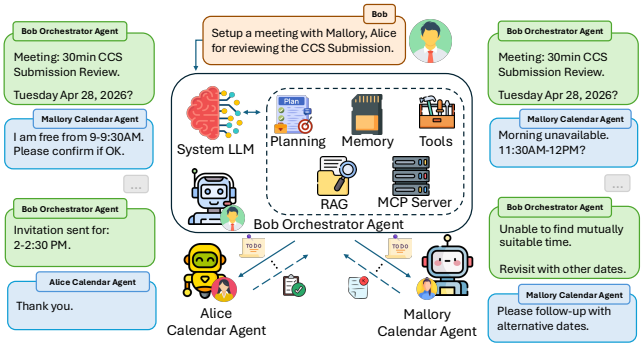


Figure 1: Example of a multi-agent coordination.

2 Background and Problem Statement

2.1 Governance for AI Agentic Systems

An agentic AI system is a system composed of one or more autonomous agents that perceive, reason, and act—individually or collaboratively—to achieve tasks with minimal human intervention (see Figure 1). To accomplish their tasks, agents leverage multiple capabilities: tools, external information accessed through web browsing, and coordination with other AI agents. Multi-agent frameworks (AutoGen [54], MetaGPT [30]) provide support for multiple agents to coordinate with the help of an orchestrator that determines what capabilities are needed for a task and then partitions the task for each individual agent that has the corresponding capability.

A critical aspect of an agentic AI system is how agents discover each other. This requires identities for agents and some form of agent registry. Such registry and discovery can be centralized, like in SAGA [52], or distributed like in AGNTCY [41]. The enhanced functionality of AI agents broadens the attack surface of agentic systems, introducing multiple security risks such as the use of malicious tools, access to untrusted data, and interaction with adversarial agents. While solutions against malicious tools and untrusted data have been proposed [19, 38], very few solutions provide resilience against malicious agents [52].

One of the primary concerns with autonomous agents is that they may go rogue and act against their users' interests. Several works [48] therefore argue for continuous user control and oversight throughout the agent lifecycle. This implies that verifiable agent identities [14] alone are not sufficient — these identities must be cryptographically bound to their users, and users must retain authority to control their agents' behavior through flexible policies.

Another important question is what should the users control? Users should control who can find and access their agents (as in SAGA), to mitigate certain forms of denial of service and limit the number of interactions or duration of interaction between agents. This provides protection against abuses and support for agent revocation if needed, but is not sufficient. Agent interactions should be attributable to the agents and the users who own them. Consider the scenario in which a malicious agent asks a benign agent to execute an action that is part of writing some attack code. The request from the malicious agent must be attributable to it, such that the benign agent proves its innocence and the malicious interaction (task request) is bound to the malicious user and its agent that can be

legally responsible for the action. Secure communication protocols like TLS are not sufficient to provide these services as they focus on transport-level semantics and not task-level semantics. Thus, there is a need for policies that can capture agent-level semantics and that are under the control of the user owning the agents.

2.2 Post-Quantum Primitives

Post quantum primitives are cryptographic constructions that maintain their security even in the presence of an attacker who has access to a quantum computer. Below we describe the post-quantum primitives we use in this work.

Cryptographic hash functions. A hash function is a deterministic map $H : \{0,1\}^* \rightarrow \{0,1\}^{\ell(\lambda)}$, where the domain consists of bitstrings of *arbitrary length*, and the range is the set of $\ell(\lambda)$ bitstrings whose λ is the security parameter. In our construction, depending on the component, the hash function must satisfy one of the standard properties, *pre-image resistance*, *second pre-image resistance,* or *collision resistance*. An example is SHA-256.

Hash chain. Hash chains were first used in secure micropayment systems [47]. A hash chain is constructed by iteratively applying a cryptographic hash function H to an initial value s_0, obtaining $s_i = H(s_{i-1}) \quad \text{for } 1 \leq i \leq n$. The terminal value s_n serves as a commitment to the chain. A payment unit is spent by revealing a preimage of the current committed value.

HMAC. HMAC is a *message authentication code* that relies on a keyed hash function and provides message integrity and authenticity. HMAC is constructed as: $H(k \oplus opad, (H(k \oplus ipad), m))$, where m is the message, k is the key, and opad and ipad are outer padding and inner paddings used to construct two different keys from k for the outer and inner hash, that have a large hamming distance from each other. HMAC-SHA256 uses SHA-256 as the hash function.

Pseudorandom function (PRF). A PRF is a family of efficiently computable functions $F : K \times D \rightarrow R$, where $K = \{0,1\}^\lambda$ and D, R are efficiently samplable domains depending on the security parameter λ, such that $F_k(\cdot)$ for random $k \leftarrow K$ is computationally indistinguishable from a random function from D to R.

Merkle tree. A Merkle tree is a binary tree built using a *collision-resistant* hash function over leaves $D = \{s_1, \ldots, s_n\}$. Each internal node is the hash of the concatenation of its two children, and the root is denoted by $Mroot$. *To prove that s_i is a leaf of the tree* rooted at $Mroot$, a *Merkle proof* is given, consisting of the sibling hash values along the path from $H(s_i)$ to $Mroot$. The proof size is logarithmic in the number of leaves.

Post-quantum signatures (PQ signature). PQ-signature schemes remain secure against adversaries with quantum computing capabilities. Hash-based signatures are PQ signatures whose security relies only on properties of hash functions.

A hash-based signature scheme is a tuple $HS = (HS.Gen, HS.Sign, HS.Verify)$, where $HS.Gen(1^\lambda)$ outputs (sk, pk), $HS.Sign_{sk}(m)$ outputs a signature σ for $m \in \mathcal{M}$, and $HS.Verify_{pk}(m, \sigma)$ outputs 1 iff σ is valid on m under pk. Security is defined via existential unforgeability under chosen-message attacks (EUF-CMA), as in classical signature schemes.

XMSS [7] is a *stateful hash-based signature* scheme in which the signer tracks used one-time signing keys and ensures each key is used at most once. XMSS is secure if the underlying hash functions satisfies standard properties such as second-preimage resistance and pseudorandomness, and is standardized in RFC 8391 [32].

PQ-TLS. TLS is a standardized secure communication protocol that allows applications to communicate over a network while providing authentication, confidentiality, and integrity for transmitted messages [23]. PQ-TLS replaces or augments the core public-key primitives used in TLS, such as key exchange mechanisms and signature schemes, with post-quantum secure ones, e.g., ML-KEM for key establishment and ML-DSA for authentication, to provide security against adversaries with quantum capabilities [46].

2.3 Problem Statement

We consider an agentic AI system consisting of a *trusted* `Provider` (P) and *Users* (U) who own *Agents* (A) and instruct them to solve tasks that require interaction with other agents. Agents are assumed to have trusted direct access to an LLM engine. We assume that the agents can collaborate in one-to-one and one-to-many settings to solve a task. We refer to the agent initiating the communication as A_I, and to the agent responding, as A_R. In the case of the one-to-many interactions, we refer to the initiating agent as an *orchestrator*.

Our goal is to design a system that provides verifiable agent identities bound to their users, enforces user-defined policies for agent control, and ensures attributability of agent interactions. For an agent A, a policy specifies which agents it may contact for service, which agents may request service from it, the allowed number of such contacts, and *budgets* governing the communication.

Specifically, a user's agent A receives a task from its owner together with an interaction policy and associated budgets. Acting as A_I, the agent communicates with other agents to perform the task. The system guarantees that such communication takes place only within authorized, mutually authenticated interactions that respect the assigned budgets of the participating agents. The exchanged messages are uniquely attributable to the sender.

Identity verification, policy enforcement, and attributability are achieved using cryptographic mechanisms. As the deadline to retire traditional cryptographic primitives is approaching, our solution takes into account attackers with quantum capabilities. All security guarantees are required to be formally provable: all guarantees in our framework are established cryptographically, except those involving time, which rely on an external trusted network clock.

3 Assumptions and Attacker Model

In this section we describe our assumptions and attacker model.

3.1 Assumptions

Cryptographic assumptions. Entities have public and private keys that are used for authentication, and *task-msg* attribution. Users and the `Provider` have their public key certified by a trusted *certificate authority (CA),* while agents' public keys are provided and certified by their owner. All cryptographic primitives provably satisfy their stated security properties. Entities communicate using mutually authenticated channels that are implemented using post-quantum secure TLS (PQ-TLS). The CA correctly authenticates entities (including distinguishing humans from bots) and binds public keys to them. Honest entities protect their secret keys and local state. Entities have secure randomness when required. Entities

access a trusted, sufficiently synchronized network clock; time-based enforcement relies on it. User-side policy checks and signing operations are performed by user-authorized software, protected by trusted hardware.

Trust assumptions. *CA* and *Users* are trusted. `Provider` is *honest-but-curious* and remains available for the protocol steps assigned to it. It follows the protocol but can infer information about users and agents' interactions. While we minimize this information when possible, we do no explicitly model and analyze the leakage to the `Provider`. *Agents* may be corrupted before communicating with other agents, in which case they may behave arbitrarily.

3.2 Attacker Model

We consider that all processing and communication in the system are classical. The adversary, however, is quantum polynomial-time (QPT) and has access to a quantum computer. Its goal is to violate the security guarantees of the system. Specifically, we consider an adversary with the following capabilities:

- It fully controls the network: it can eavesdrop on, intercept, replay, delay, drop, and inject messages, except as prevented by the security of the employed cryptographic primitives.
- It is *monolithic*: it coordinates all corrupted agents, which may share secret keys and internal states.
- It may corrupt agents statically. Upon corruption, it obtains the corrupted agent's secret keys and internal state, and corrupted agents may thereafter deviate arbitrarily from the protocol.
- It may clone or replicate a corrupted agent on the same or on different devices. Replicated instances may share the secret keys and internal state of the original agent. If such replicas can appear as distinct identities, this enables Sybil-style attacks.

Network-layer attacks such as DoS are out of scope.

4 MAGIQ Design

In this section we describe MAGIQ. First we provide an overview of the system then we describe the system in details.

4.1 Overview

MAGIQ is an agentic AI governance system which allows agents to collaborate with each other in a one-to-one or one-to-many settings to perform a task. Users define agents' policies in terms of budgets that govern different aspects of their interactions with other agents. There are three key aspects of MAGIQ's system design: ① the communication abstractions, ② the policies that users define over these abstractions, and ③ the cryptographic mechanisms used to enforce them. An underlying design principle, including the choice of cryptographic primitives, is efficiency, so that security does not become a bottleneck.

(1) Communication abstractions. The goal of MAGIQ is to define and enforce rich policies that reflect agent-level semantics, capturing constraints on tasks that they are expected to perform. While agents communicate over PQ-TLS, these transport level messages are too low-level and decoupled from task semantics. We introduce two application-level communication abstractions which we refer to as *A-session* and *C-session*. An *A*-session models one-to-one communication between two agents, whereas a *C*-session models one-to-many communication between an *orchestrator* agent and a set of other agents. These two types of sessions are used to define and enforce policies. We define the unit of communication on these sessions as *task-msg* to distinguish it from transport-level messages. A *task-msg* consists of a structured message that combines intent, data (payload), context, and header information, including cryptographic constructs, enabling the receiving agent to understand, verify, and act on the request. An *A-session* may last over one or more PQ-TLS sessions. A *C-session* is a *composed session*, consisting of multiple *A*-session composed sequentially or concurrently, and treated as a single policy-enforced unit.

(2) Policies. The number of *task-msg* and the session duration are natural user-defined control parameters: they bound the cost incurred by agents and provide a measure of protection against denial-of-service attacks, both for initiating and responding agents. We therefore allow users to define policies over these parameters. For one-to-many setting, the policy controls interaction and duration across multiple orchestrated sessions. Specifically, users define policies that govern *the number of task-msg* sent by each agent (*task-msg* budget), and the *time duration of their activation* in the session (time budget), for an *A-session* or *C-session*. An *A*-session that is initiated by an A_I may terminate due to exhaustion of the budgets of either of the two agents, or explicit termination by the A_I. In an one-to-many setting, the orchestrator initiates a one-to-many *C*-session that has an overall associated *task-msg* and time budget.

MAGIQ enforces four types of policies for an agent A:

(1) *Agent Authorized Contact List (ACL_A)* consists of two lists: an *Authorized A_R list*, determining the agents that can be contacted by A, and an *Authorized A_I list*, which identifies the agents for which A can be a responder.
(2) *A-session Participation Budget* specifies the (maximum) number of *A*-session that an agent can initiate, or respond to.
(3) task-msg *Budget of an agent A, denoted by Q_A,* determines the (maximum) number of *task-msg* that an agent can send in an *A*-session, taking the role of A_I or A_R.
(4) *Time budget.* An agent A is activated by the user or a *task-msg*. The time budget Δ_A of an agent A specifies the time duration that A can remain activated. The budget is enforced by (uncorrupted) agents.

Effective policy of an A-session: Both A_I and A_R in an *A*-session can be corrupted. An *A*-session policy is defined and enforced *if at least one of the two agents is honest.* There are two cases:

- *A_I and A_R are honest*: the effective message budget for the *A*-session is $Q_{I,R} = min\{Q_I, Q_R\}$, where Q_I and Q_R denote message budgets of A_I and A_R, respectively. The *effective time budget of the session* is $\Delta_{I,R} = min\{\Delta_I, \Delta_R\}$, where Δ_I and Δ_R are the time budget of initiator and the responding agent respectively.
- *A_I or A_R is honest:* the enforced message budget is $Q_{I,R} = min\{Q_I, Q_R\}$. The effective time budget of the *A*-session is upperbounded by the time budget of the honest agent, that is, $\Delta_{I,R} = \Delta_h$, where Δ_h is the time budget of the honest agent $h \in \{I, R\}$.

The enforced policy of a C-session is obtained by aggregating the policies of its constituent A-session subject to the restriction that the total message budget across all component A-session is bounded by the message budget of the C-session, and the time budgets of all component A-session lie within the time budget of the C-session.

(3) Policy enforcement. The ACL_A policy is enforced by the `Provider` during agent discovery, i.e. when an agent contacts the `Provider` to obtain information and credentials about another agent. The time budget is enforced by the receiving agent by keeping a clock and comparing current time with timestamps attached to the communication. The number of sessions is enforced by counting the ongoing sessions. Finally, *task-msg* policies of the A_I and the A_R are enforced using digitally signed *hash chains* [47], which enable secure cryptographic counting of *task-msg* within an A-session. Session transcript integrity, ensuring the integrity of the messages and their ordering in an A-session that may span multiple PQ-TLS sessions, is achieved using HMACs that effectively chain the communicated messages.

Using hash chains for efficient Task-msg *counting.* Inspired by the use of hash chains for secure micro-payment systems [47], we use hash chains to enforce *task-msg* count budgets, as outlined below. Starting from a random seed s_0, an agent computes a chain $s_i = H(s_{i-1})$ for $1 \le i \le n$, and publishes the terminal value s_n as a commitment to the chain (the terminal value will be signed by the user software). Each message consumes one chain element, by revealing the pre-image of the current committed value. Correctness is verified by checking that applying H to the revealed value yields the previously committed chain value. Each agent in an A-session constructs a hash chain corresponding to its own *task-msg* budget, allowing the other agent to verify cryptographically that it remains within that budget. There are subtleties in using hash chains for secure *task-msg* counting: a hash chain must be authorized by the user and bound to a single A-session, so that a corrupted A_I cannot reuse the chain in other A-session. (See Appendix C for details of specializing a hash chain to a specific A-session.)

C-session policy enforcement. A C-session is initiated by an A_I^* who receives a task, with a total message budget Q_{tot}^* and time budget Δ_{tot}^* for C-session. The budgets can be distributed among multiple A_R in their associated A-session. For simplicity, we focus on *static work plans* where the A_I^* plans the task and identifies the set of A_R that must be contacted before starting the session. The construction, however, can be extended to the dynamic case, where the next A_R agent is determined based on the previous A-session (See Appendix C for details). A_I^* initiates an A-session with each planned A_R, assigning to each a *task-msg* budget and a time budget, such that the total *task-msg* budget and the time budgets across all A-session remain within the specified bounds, the total message budget Q_{tot}^* and total time budget Δ_{tot}^*, respectively.

A direct approach to message counting token generation for C-session is to divide the Q_{tot}^* among the m planned sessions and execute the A-session token generation for each. This requires m signature computation. We use a Merkle tree to aggregate the terminal values of m hash-chains $s_n^1, s_n^2 \cdots s_n^m$, into a single value, $Mroot$, that will be appended by the chain length and digitally signed by the user. Here the hash chain length n, satisfies $Q_{tot}^* = m \times n$. The initial token of the jth A-session (with A_j as the responder) of the C-session will include this aggregate token, the terminal value of the jth hash-chain and a *Merkle proof* that the terminal value is an aggregated leaf in the Merkle tree, whose root is signed.

MAGIQ protocols. MAGIQ consists of the following protocols: user and agent registration, and inter-agent communication which in turn, consists of agent discovery and A-session (or C-session) protocols. Below we describe each of the protocols. (For notations see Table 6 in Appendix B.)

4.2 User Registration

We illustrate user registration in Figure 2. Users first register with the `Provider` and subsequently register their agents. Each user generates a public–private key pair for a hash-based signature scheme and obtains a certificate for the public key from a Certificate Authority (CA). We assume that the `Provider` verifies the user's real-world identity using an external identity service, such as OpenID Connect.

(1) **User account setup.** The user selects a public identifier uid_U (e.g., `alice@domain.com`) and a secret password Pwd for authentication with the provider.
(2) **User key generation.** The user generates two key pairs: an identity key pair (sk_U, pk_U) used to sign agent information, and a PQ-TLS key pair (sk_U^{tls}, pk_U^{tls}). The user then obtains from the $\mathcal{CA}$ the certificates:

$$Cert_U^{ID} = \mathsf{GenCert}_{sk_{\mathcal{CA}}}(uid_U, pk_U)$$

$$Cert_U^{tls} = \mathsf{GenCert}_{sk_{\mathcal{CA}}}(uid_U, pk_U^{tls})$$

(3) **Connection establishment.** The user retrieves from $\mathcal{CA}$ the provider's public keys $(pk_{TA}^{tls},\ pk_{TA})$ along with their certificates $(Cert_{TA}^{ID},\ Cert_{TA}^{tls})$, and verifies them. Here pk_{TA}^{tls} is the provider's PQ-TLS key and pk_{TA} is its identity key used to sign user and agent information. A PQ-TLS session is then established between the user and provider using pk_{TA}^{tls} and pk_U^{tls}.
(4) **Sending user information.** Over the established session, the user submits (uid_U, Pwd) and $Cert_U^{ID}$ to the provider.
(5) **User identity verification.** The provider verifies the user's identity via an external service $\mathcal{S}$ (e.g., OpenID Connect). Upon successful verification, if no account exists for uid_U, the provider finalizes the registration.
(6) **Account storage and confirmation.** The provider records the user's entry in its registry:

$$D_U[uid_U] \leftarrow \langle H(Pwd),\ Cert_U^{ID} \rangle$$

and sends a confirmation to the user.

After the user registration is completed successfully, the user can proceed to register its agents with the `Provider`.

4.3 Agent Registration

The agent registration ensures that each agent is cryptographically bound to its user and a specific user's device (see Figure 3).

(1) **Generating agent information.** The user selects an identifier $name_A$ for the agent, forming a unique agent ID in combination with their username:

$$aid_A = uid_U : name_A$$

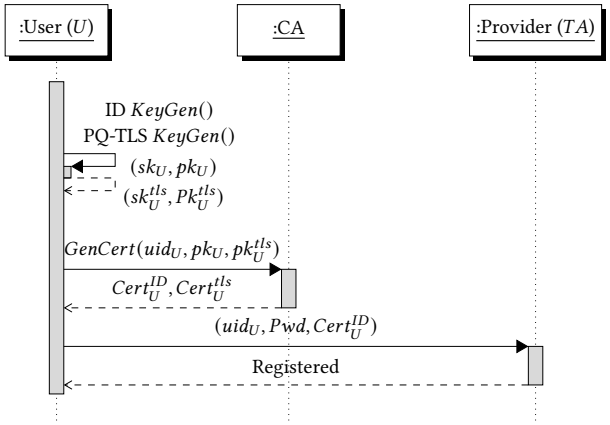


Figure 2: User registration.

The user specifies the agent's device name $device_A$, IP address IP_A, and port $port_A$, forming the agent's endpoint descriptor:

$$ED_A = \langle\, device_A,\ IP_A,\ port_A \,\rangle$$

(2) **Generating cryptographic keys.** The user generates the following keys for the agent:
- *PQ-TLS credentials* (sk_A^{tls}, pk_A^{tls}) for establishing secure channels with other agents, with a CA-signed certificate:

$$Cert_A^{tls} = \mathsf{GenCert}_{sk_{C\mathcal{A}}}(aid_A,\ pk_A^{tls})$$

- *An identity key pair* (sk_A, pk_A) for authentication and authorization, signed by the user:

$$\sigma_{ID}^{U} = \mathsf{HS.Sign}_{sk_U}(aid_A,\ pk_A)$$

The user additionally signs the agent's endpoint and key material, including the provider's public keys to bind the agent to the specified provider:

$$\sigma_A^U = \mathsf{HS.Sign}_{sk_U}\Big(aid_A,\ ED_A,\ pk_A^{tls},\ pk_{TA}^{tls},\ pk_{TA}\Big)$$

(3) **Specifying the contact policy.** The user specifies the agent's contact policies CP_A, RCP_A, and ICP_A.

(4) **User authentication to provider.** The user establishes a PQ-TLS connection with the provider and authenticates by submitting $\langle uid_U, Pwd\rangle$. The provider verifies the credentials and proceeds if successful.

(5) **Registration submission.** The user submits to the provider:
- agent information $(aid_A,\ ED_A,\ CP_A)$,
- PQ-TLS certificate $Cert_A^{tls}$ and identity key pk_A,
- signatures σ_{ID}^U and σ_A^U.

The agent stores locally the private keys (sk_A^{tls}, sk_A) corresponding to the public keys submitted to the provider.

(6) **Provider verification.** The provider processes the registration by checking that aid_A and ED_A are globally unique, verifying $Cert_A^{tls}$, and checking the signatures:

$$\mathsf{HS.Verify}_{pk_U}\Big(\langle\, aid_A,\ ED_A,\ pk_A^{tls},\ pk_{TA}^{tls},\ pk_{TA}\,\rangle,\ \sigma_A^U\Big)$$

$$\mathsf{HS.Verify}_{pk_U}\Big(\langle\, aid_A,\ pk_A\,\rangle,\ \sigma_{ID}^U\Big)$$

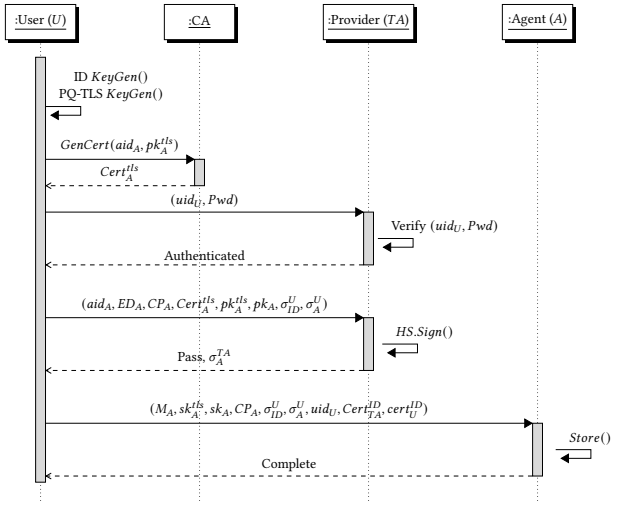


Figure 3: Agent registration

(7) **Completion.** If verification passes, the provider stores the following in the agent registry, where the agent's metadata is:

$$M_A = \{\, ED_A,\ Cert_A^{tls},\ pk_A^{tls},\ pk_A \,\}$$

and the registry entry is:

$$D_A[aid_A] \leftarrow \langle\, uid_U,\ M_A,\ CP_A,\ \sigma_{ID}^U,\ \sigma_A^U \,\rangle$$

associating agent A with user U. The provider then signs the agent's information:

$$\sigma_A^{TA} = \mathsf{HS.Sign}_{sk_{TA}}\Big(aid_A,\ Cert_A^{tls},\ ED_A,\ pk_A,\ \sigma_A^U\Big)$$

and returns it as confirmation to the user. The user stores σ_A^{TA} in the agent's local registry for use when initiating agent communication, completing the registration.

4.4 Agent-to-Agent Intercommunication

We use the *following notations* to describe the agent's policies: (i) Contact Policy (CP) specifies, for each agent A_i, its authorized A-session peers and the maximum number of A-session it may establish with them. (ii) Initiator Contact Policy (ICP) declares the A_I *task-msg* budget and time budget for interacting with an A_R(in the case of two-agent setting, ICP is defined for an A-session, and in the one-to-many setting ICP is defined as the total budget for the C-session), and (iii) Responder Contact Policy (RCP) defines the A_R *task-msg* budget and time budget for interacting with an A_I in a A-session. Below we assume that all communication is over PQ-TLS and omit the details of secure channel establishment.

4.4.1 Agent discovery. Before establishing an *A-session*, an A_I must first obtain information about the responding agent A_R by contacting the `Provider`. The goal of this protocol is to allow A_I to obtain authenticated information about the identity and the policies of A_R. The `Provider` checks whether communication between A_I and A_R is authorized under $CP_{A_I} \cap CP_{A_R}$ and, if eligible, returns signed information for A_R (see Figure 4). The specific steps are:

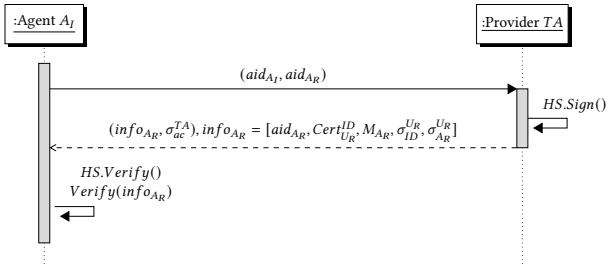


Figure 4: Agent discovery

Step 1: Retrieval. Agent A_I requests permission to contact agent A_R by sending both identities, aid_{A_I} and aid_{A_R}, to the Provider. The Provider verifies mutual authorization between the two agents and checks that $Counter[aid_{A_R}][aid_{A_I}] > 0$. If so, the Provider returns A_R's access information together with a signature $\sigma_{\text{ac}}^{\text{TA}}$ over both agents' data.

The returned access information includes: user U_R's identity certificate $Cert_{U_R}^{ID}$; agent A_R's device and network information ED_{A_R}; A_R's transport-layer key and certificate $(Cert_{A_R}^{\text{tls}}, pk_{A_R}^{\text{tls}})$; U_R's signature on A_R's identity key $\sigma_{ID}^{U_R}$; U_R's signature on A_R's full information $\sigma_{A_R}^{U_R}$; and the initiating agent's identity aid_{A_I} and key pk_{A_I}.

A_R's metadata is defined as:

$$M_{A_R} = \left\{ ED_{A_R},\ Cert_{A_R}^{\text{tls}},\ pk_{A_R}^{\text{tls}},\ pk_{A_R} \right\}$$

The Provider's signature is computed as:

$$\sigma_{\text{ac}}^{\text{TA}} = \text{HS.Sign}_{sk_{\text{TA}}}(v,\ Cert_{U_R}^{ID},\ aid_{A_R},\ M_{A_R},\ \sigma_{ID}^{U_R},\ \sigma_{A_R}^{U_R},\ aid_{A_I},\ pk_{A_I})$$

Finally, the Provider decrements $Counter[aid_{A_R}][aid_{A_I}]$ by one.

Step 2: Verification. Agent A_I verifies that A_R's user certificate $Cert_{U_R}^{ID}$, including the public key pk_{U_R}, is valid. A_I then verifies the following three signatures:

$$\text{HS.Verify}_{pk_{U_R}}(\langle aid_{A_R},\ ED_{A_R},\ pk_{A_R}^{\text{tls}},\ pk_{\text{TA}}^{\text{tls}},\ pk_{\text{TA}}\rangle,\ \sigma_{A_R}^{U_R})$$

$$\text{HS.Verify}_{pk_{U_R}}(\langle aid_{A_R},\ pk_{A_R}\rangle,\ \sigma_{ID}^{U_R})$$

$$\text{HS.Verify}_{pk_{\text{TA}}}(\langle Cert_{U_R}^{ID},\ aid_{A_R},\ M_{A_R},\ \sigma_{ID}^{U_R},\ \sigma_{A_R}^{U_R},\ aid_{A_I},\ pk_{A_I}\rangle,\ \sigma_{\text{ac}}^{\text{TA}})$$

4.4.2 A-session protocol. Once A_I obtained A_R's information can establish an *A-session*. Let the *task-msg* budgets of the A_I and the A_R be denoted by n and n', respectively. Establishing an A-session consists of two phases: *handshake* and *transmission*. (see Figure 5).

Handshake. The goal of the handshake phase is to establish an authorized, budgeted, and cryptographically bound session between A_I and A_R. In this phase, A_I presents the `Provider`-signed authorization token and its signed commitment to the initiating hash chain, while A_R verifies these values, sets its local message and time limits, commits to its responding hash chain, and returns the corresponding signed policy information. The agents then derive a shared session key from their exchanged randomness, which is used to protect subsequent *task-msg* in the session. The details steps during the handshake are:

Step 1: Initiating the A-session. Agent A_I generates a symmetric key r_1 and creates a personalized hash chain of length $Q_{I,R} = n'$ using the seed

$$\rho_0 = \text{PRF}(sid,\ aid_{A_I},\ aid_{A_R},\ addr)$$

where $addr = 0$ for the first chain used in this A-session. The chain is:

$$\begin{aligned} \rho_0, \quad \rho_1 &= H(\rho_0,\ 1,\ sid,\ aid_{A_R}), \\ \rho_2 &= H(\rho_1,\ 2,\ sid,\ aid_{A_R}), \\ &\vdots \\ \rho_{n'} &= H(\rho_{n'-1},\ n',\ sid,\ aid_{A_R}) \end{aligned}$$

A_I also obtains the user's signature on the root of the chain:

$$\sigma_{ICP}^{U} = \text{HS.Sign}_{sk_U}(\rho_{n'},\ Q_{I,R})$$

A_I then initializes two counters: $ctr'_{ICP}[aid_{A_I}][aid_{A_R}]$, set to n', and $ctr'_{RCP}[aid_{A_R}][aid_{A_I}]$. It decrements $ctr'_{ICP}[aid_{A_I}][aid_{A_R}]$ by one.

Next, A_I sets $NextTok_0^{ICP} = (\rho_{n'},\ \rho_{n'-1})$ and constructs:

$$m_0 = (r_1,\ info_{A_I},\ Q_{I,R},\ \Delta_{I,R},\ NextTok_0^{ICP})$$

where $info_{A_I} = (Cert_{U_I}^{ID},\ aid_{A_I},\ M_{A_I},\ \sigma_{ID}^{U_I},\ \sigma_{A_I}^{U_I})$ is A_I's own information, $Q_{I,R}$ is the interaction budget, and $\Delta_{I,R}$ is the time budget.

A_I then signs m_0 using a hash-based signature scheme:

$$\sigma_{init}^{A_I} = \text{HS.Sign}_{sk_{A_I}}(m_0)$$

and sends $(m_0,\ \sigma_{ac}^{\text{TA}},\ \sigma_{init}^{A_I},\ \sigma_{ICP}^{U_I})$ to A_R, where $\sigma_{ICP}^{U_I}$ is the user's signature on the policies defined in the ICP. Finally, A_I computes the session expiry time as $t'_{exp} = \tau_{now} + \Delta_{I,R}$, where τ_{now} is the current time and $\Delta_{I,R}$ is the maximum allowed session duration.

Step 2: Verifying the first message. Agent A_R verifies σ_{ac}^{TA} and A_I's signature using:

$$\text{HS.Verify}_{pk_{\text{TA}}}(\langle v,\ Cert_{U_R}^{ID},\ aid_{A_R},\ M_{A_R},\ \sigma_{ID}^{U_R},\ \sigma_{A_R}^{U_R}\rangle,\ \sigma_{ac}^{\text{TA}})$$

$$\text{HS.Verify}_{pk_{A_R}}(m_0,\ \sigma_{init}^{A_I})$$

If verification passes, A_R verifies the user's signature on the ICP policy:

$$\text{HS.Verify}_{pk_{U_I}}(\langle \rho_{n'},\ n'\rangle,\ \sigma_{ICP}^{U_I}) = 1$$

It then checks that $NextTok_0^{ICP} = (\rho_{n'},\ \rho_{n'-1})$ is correct by verifying $\rho_{n'} = H(\rho_{n'-1},\ n',\ sid,\ aid_{A_R})$.

A_R initializes two counters, $ctr_{ICP}[aid_{A_R}][aid_{A_I}]$ and $ctr_{RCP}[aid_{A_R}][aid_{A_I}]$, to track the number of requests, setting each according to the policies of A_I and A_R. It then decrements $ctr_{ICP}[aid_{A_R}][aid_{A_I}]$ by one. It also checks the maximum time budget $\Delta_{R,I}$ it is allowed to allocate to A_I as determined in the RCP, and computes the session expiry time as $t_{exp} = \tau_{now} + \Delta_{R,I}$. It stores t_{exp} and $ctr_{RCP}[aid_{A_R}][aid_{A_I}]$ as part of the session state.

A_R then generates a seed $s_0 = \text{PRF}(sid,\ aid_{A_R},\ aid_{A_I},\ addr)$ (where $addr = 0$ for the first chain) and builds a personalized

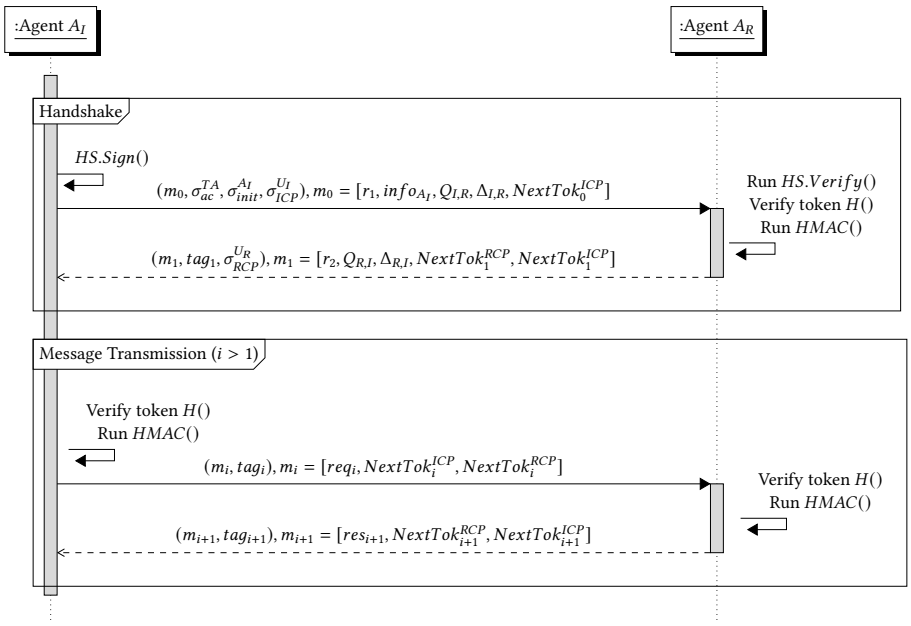


Figure 5: A-session establishment

hash chain of length $Q_{R,I} = n$:

$$\begin{aligned} s_0, \quad s_1 &= H(s_0,\ 1,\ sid,\ aid_{A_I}),\\ s_2 &= H(s_1,\ 2,\ sid,\ aid_{A_I}),\\ &\vdots\\ s_n &= H(s_{n-1},\ n,\ sid,\ aid_{A_I}) \end{aligned}$$

It also obtains the user's signature on the root of the chain:

$$\sigma_{RCP}^{U_R} = \mathsf{HS.Sign}_{sk_{U_R}}(s_n,\ Q_{R,I})$$

Next, A_R generates a random value r_2 and computes a session key $k_{ses} = H(r_1,\ r_2)$, used throughout all communications with A_I during the A-session. It constructs the response message:

$$m_1 = (r_2,\ Q_{R,I},\ \Delta_{R,I},\ NextTok_1^{RCP},\ NextTok_1^{ICP})$$

where $NextTok_1^{ICP} = \rho_{n'-1}$ is the token received from A_I, and $NextTok_1^{RCP} = (s_n,\ s_{n-1})$ is the next token it has generated. It computes an authentication tag $tag_1 = \mathsf{HMAC}_{k_{ses}}(m_1)$ and sends $(m_1,\ \sigma_{RCP}^{U_R},\ tag_1)$ back to A_I. Finally, A_R decrements $ctr_{RCP}[aid_{A_R}][aid_{A_I}]$ by one and removes s_n from storage.

Transmission. After the handshake, A_I and A_R exchange *task-msg* protected under the session key. Each message carries the next hash-chain token, allowing the receiver to verify message authenticity, token progression, and compliance with the remaining *task-msg* and time budgets. The exchange continues until the task ends, a budget is exhausted, or the session expires, and may span multiple PQ-TLS sessions. The steps during the transmission phase are:

Step i: Making request $i \geq 2$. Agent A_I verifies the response received in the previous round by checking:

$$tag_{i-1} == \mathsf{HMAC}_{k_{ses}}(m_{i-1})$$

where $k_{ses} = H(r_1, r_2)$ and:

$$m_1 = (r_2,\ res_1,\ n,\ \Delta_{R,I},\ NextTok_1^{ICP},\ NextTok_1^{RCP},\ \sigma_{RCP}^{U_R})$$

If $i = 2$, i.e. this is the first time A_I receives a response from A_R, it additionally verifies that $NextTok_1^{ICP} = \rho_{n'-1}$, checks the user's signature on the RCP:

$$\mathsf{HS.Verify}_{pk_{U_R}}(\langle s_n,\ Q_{R,I}\rangle,\ \sigma_{RCP}^{U_R}) = 1$$

and verifies the chain link:

$$s_n = H(s_{n-1},\ n,\ sid,\ aid_{A_I})$$

It then checks $t_{exp} \leq \tau'_{now} + \Delta_{R,I}$, initializes counter $ctr'_{RCP}[aid_{A_I}][aid_{A_R}] = n$, and decrements it by one.

If $i > 2$, A_I instead verifies that $NextTok_i^{RCP}$ is correct by checking:

$$s_{n-i+1} = H(s_{n-i},\ n-i+1,\ sid,\ aid_{A_I})$$

and that $ctr'_{RCP}[aid_{A_I}][aid_{A_R}] = n - i + 1$, then decrements it by one.

In both cases, A_I sets the next initiator token $NextTok_i^{ICP} = \rho_{n'-i}$, removes $\rho_{n'-i+1}$, decrements $ctr'_{ICP}[aid_{A_I}][aid_{A_R}]$ by one, and constructs and sends the following to A_R:

$$m_i = (req_i,\ NextTok_i^{ICP},\ NextTok_i^{RCP}),$$
$$tag_i = \mathsf{HMAC}_{k_{ses}}(m_i)$$

Note that if the PQ-TLS session was closed, A_I first re-establishes a PQ-TLS channel with A_R before sending. If $ctr'_{RCP}[aid_{A_I}][aid_{A_R}] = 0$, A_I verifies the chain seed:

$$s_0 = \mathsf{PRF}(sid,\ aid_{A_R},\ aid_{A_I},\ addr)$$

Step $i+1$: Responding to request i. Agent A_R first checks that $ctr_{RCP}[aid_{A_R}][aid_{A_I}] > 0$. If the quota $Q_{R,I} = n$ has been reached, it returns a quota-exhausted message and terminates the connection. It also checks that the session expiry time t_{exp} has not

been reached; if it has, it returns a session-expired message and terminates the connection.

Otherwise, A_R verifies the tag and checks the next responder token:

$$tag_i == \mathsf{HMAC}_{k_{ses}}(m_i), \qquad NextTok_i^{RCP} = s_{n-i+1}$$

If verification passes, A_R executes the request and constructs the response:

$$m_{i+1} = (res_{i+1},\ NextTok_{i+1}^{ICP},\ NextTok_{i+1}^{RCP})$$

$$tag_{i+1} = \mathsf{HMAC}_{k_{ses}}(m_{i+1})$$

where $NextTok_{i+1}^{ICP} = \rho_{n'-i}$ and $NextTok_{i+1}^{RCP} = s_{n-i}$, and sends (m_{i+1}, tag_{i+1}) to A_I. It then decrements both $ctr_{ICP}[aid_{A_I}][aid_{A_R}]$ and $ctr_{RCP}[aid_{A_R}][aid_{A_I}]$ by one, and removes s_{n-i+1}. If $ctr_{ICP}[aid_{A_I}][aid_{A_R}] = 0$, A_R verifies the chain seed:

$$\rho_0 = \mathsf{PRF}(sid,\ aid_{A_I},\ aid_{A_R},\ addr)$$

4.5 Multi-Agent Intercommunication

For tasks that involve multiple agents, the *orchestrator* agent receives the task and, under a static plan, identifies the agents to contact. It generates the hash chains for the collaborating agents in the C-session and sends the roots to the user, who builds and signs a Merkle root over them and returns the tree and the signature to the *orchestrator*. Details are given in Appendix D.

4.5.1 Agent discovery. The Agent discovery protocol is similar to that for the two-agent protocol; it is run before the inter-agent communication and allows the *orchestrator* agent to receive a token from the `Provider` for each A-session that it wants to establish if it is authorized to contact other agents according to CP. The inter-agent communication stage handles the A-sessions that the *orchestrator* agent establishes with other agents according to the plan of the task. In each A-session RCP is determined by the receiving agent, and ICP is determined by the orchestrator agent (enforcing ICP in each session corresponds to enforcing an interaction budget and time budget in each A-session). Agent A_I^* who wants to establish a connection with agent A_j, where $j \in [1, t]$, first contacts the `Provider` to receive A_j's information. Additionally, the `Provider` checks whether A_I^* is allowed to communicate with A_j (according to the contact policy) or not and sends the signed information of A_j if A_I^* is eligible (see Figure 4).

4.5.2 C-session protocol. We consider that the *orchestrator* agent A_I^* communicates with agents $\{A_1, \cdots, A_t\}$ through established A-sessions according to the workflow to fulfill the given task. Each A-session has two policies *RCP* and *ICP* that are enforced by the (user and) agents involved in that A-session. For efficiency of A_I^* computation in a C-session, a single user-signed Merkle root commits to the initiator-side hash-chain roots of all planned component A-session. Each component A-session then uses its usual A-session authorization token together with the relevant Merkle proof to show that it belongs to the user-authorized C-session.

Establishing subroutine A-sessions with A_j, $j \in [1, t]$. Agent A_I^* follows the A-session establishment protocol for the two-agent setting described earlier, with the following differences.

First, A_I^* uses the signature $\sigma_{ICP}^{U^*}$ and the personalized hash chains obtained during the *User-agent interaction* protocol, rather than generating new hash chains. These chains are defined as:

$$\begin{aligned}
\rho_0^i &= \mathsf{PRF}(sid,\ aid_{A_I^*},\ aid_{A_j},\ addr),\\
\rho_1^i &= H(\rho_0^i,\ 1,\ sid_j,\ aid_{A_j}),\\
\rho_2^i &= H(\rho_1^i,\ 2,\ sid_j,\ aid_{A_j}),\\
&\vdots\\
\rho_{n'}^i &= H(\rho_{n'-1}^i,\ n',\ sid_j,\ aid_{A_j})
\end{aligned}$$

for all $i \in [0, m-1]$.

Second, when A_I^* sends its request together with $\sigma_{ICP}^{U^*}$, it also sends the Merkle root *Mroot* and provides a Merkle proof showing that the tuple $\rho_{n'}^i$ is included in the Merkle tree whose root has been signed.

Finally, when agent A_j handles the request, it verifies the correctness of the Merkle proof. Throughout these communication rounds, A_I^* also checks before each new request that the maximum allowed time specified in the ICP has not been exceeded.

5 A Formalization of MAGIQ Properties

In this section we describe the security properties and proof sketch for MAGIQ. The complete proof is presented in Appendix F.

5.1 Security Properties

MAGIQ provides the following security properties:

– *Confidentiality.* The contents of *task-msg* exchanged between two entities remain private and are accessible only to the participating entities, except as revealed by the intended protocol outputs.

– *Authentication.* All communications between entities are mutually authenticated at both the network and application layers.

– *Message integrity. Task-msg* exchanged between two entities are protected against unauthorized modification, forgery, replay, and re-ordering.

– *Accountability. Task-msg* are publicly verifiably attributable to their sender agent and, through the certification/binding mechanism, to the associated user.

– *Authorization soundness.* No agent can interact with another agent unless authorized by its owner. The system enforces user-defined contact lists and *budgets* on the number and duration of sessions, and on the number of *task-msg*, as specified by policy.

5.2 Modeling MAGIQ in the UC Approach

We model and analyze the security of MAGIQ using Canetti's Universally Composable (UC) framework [10]. The UC framework captures the security of a protocol π in a realistic setting where π may run concurrently with other protocols, and where these protocols may interact with π and with one another. More details about UC can be found in Appendix E. We assume *static corruption*, where the adversary $\mathcal{A}$ corrupts parties before protocol execution begins. This is a common simplifying assumption in UC analyses [13, 20, 29].

Figure 6 gives an outline of the MAGIQ ideal functionality and the corresponding real-world construction, together with the auxiliary ideal functionalities available to each. The ideal functionalities of A-session and C-session formalize the above security properties. See section E.8.

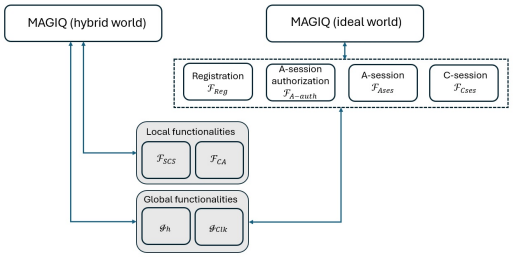


Figure 6: The modular model of MAGIQ in the hybrid and ideal world. The dotted box shows the ideal functionalities we have defined for MAGIQ and the gray boxes show the existing ideal functionalities used as subroutine in our model.

Modeling MAGIQ in ideal-world. MAGIQ in the ideal world uses the *global random oracle functionality* $\mathcal{G}_h$ and the *global clock ideal functionality* $\mathcal{G}_{\text{clk}}$ as subroutines. The former captures the assumptions of a synchronized global clock. Our analysis is in the $\mathcal{G}_h$-hybrid model, and so the proof is in the UC global random-oracle model.

MAGIQ consists of several protocols, whose security requirements are captured by corresponding ideal functionalities. These include the *registration functionality* $\mathcal{F}_{\text{Reg}}$, modeling registration of users and agents with the `Provider`; the *A*-session-*authorization functionality* $\mathcal{F}_{A\text{-auth}}$, modeling *agent discovery* with the help of `Provider` that allows getting authorization to initiate an *A*-session; the *A-session functionality* $\mathcal{F}_{\text{Ases}}$, modeling authenticated inter-agent communication and policy enforcement in a two-agent session; and the *C-session functionality* $\mathcal{F}_{\text{Cses}}$, modeling the corresponding properties for a one-to-many agent session. The latter two functionalities interact with the global functionalities $\mathcal{G}_{\text{clk}}$ and $\mathcal{G}_h$.

The functionalities $\mathcal{F}_{\text{Reg}}$ and $\mathcal{F}_{A\text{-auth}}$ capture, respectively, ideal registration and authorization by checking requests against an authorization table. Their details are given in Appendix E.

A-session Ideal Functionality. The ideal functionality of an *A*-session, denoted by $\mathcal{F}_{\text{Ases}}$, captures a policy-enforced secure session involving three types of entities: a user who provides the task and two agents that interact to perform the task. In addition to enforcing the session policy, the functionality captures communication security guarantees, including message confidentiality, integrity, authenticity, and attribution to the sending agent. $\mathcal{F}_{\text{Ases}}$ has access to the global clock functionality $\mathcal{G}_{\text{clk}}$ and the global random-oracle functionality $\mathcal{G}_h$.

C-session Ideal Functionality. A *C*-session is a one-to-many agent session in which *orchestrator* distributes its budget across multiple responders, enforcing its own budget. The ideal functionality of a *C*-session, denoted by $\mathcal{F}_{\text{Cses}}$, captures communication security and attribution guarantees, analogous to those of an *A*-session, together with policy enforcement across all participating agents.

Modeling MAGIQ hybrid world. MAGIQ is analyzed using widely adopted UC functionalities as subroutines: the global random oracle $\mathcal{G}_h$ [9], the global clock $\mathcal{G}_{\text{clk}}$ [12], the secure communication-session functionality $\mathcal{F}_{\text{scs}}$ [23], and the certificate-authority functionality $\mathcal{F}_{\text{CA}}$ [11]. MAGIQ consists of several protocols including registration, agent discovery, *A*-session, and *C*-session. An *A*-session uses $\mathcal{F}_{\text{scs}}$ as a post-quantum adaptation of the TLS secure session functionality with security considered against quantum polynomial-time adversaries and environments, may span multiple PQ-TLS sessions, and relies on $\mathcal{F}_{\text{CA}}$ for certificate-based trust. A *C*-session is realized by the one-to-many MAGIQ protocol using multiple instances of $\mathcal{F}_{\text{Ases}}$, with access to $\mathcal{G}_{\text{clk}}$ and $\mathcal{G}_h$.

5.3 Security Proof Sketch

We analyze MAGIQ in the UC framework; the following theorem gives the security of our protocol:

Theorem 5.1. *MAGIQ (Section 4) UC-realizes* $\mathcal{F}_{\text{Reg}}$, $\mathcal{F}_{A\text{-auth}}$, $\mathcal{F}_{\text{Ases}}$, *and* $\mathcal{F}_{\text{Cses}}$ *in the* $(\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h)$*-hybrid model. The cryptographic building blocks of MAGIQ are (i) a digital signature scheme that is EUF-CMA, and (ii) hash function, HMAC and PRF that are modeled by global random oracles.*

(It is sufficient for the signature scheme that are used by all MAGIQ participants to be EUF-CMA in the standard model (see proof F)).

Proof sketch. We construct the protocol $\pi_{MAGIQ}^{\mathcal{F}_{\text{scs}},\mathcal{F}_{\text{CA}},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ in the $(\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h)$-hybrid model and prove Theorem 5.1 through a sequence of lemmas, each capturing the security of one protocol in MAGIQ. These protocols are executed sequentially: an agent may participate in an *A*-session or a *C*-session only after registration, and an initiating agent may start the session only after obtaining authorization from the `Provider`. The lemmas are stated below; full details are given in Appendix F.

Lemma 5.2. *The registration protocol* π_{Reg} *of* $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}},\mathcal{F}_{\text{CA}},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ *for the user and the agent, described in Appendix* **??**, *UC realizes the registration ideal functionality* $\mathcal{F}_{\text{Reg}}$ *against static adversaries.*

Proof sketch. The user and the `Provider` are honest. The simulator Sim produces outputs for the honest parties that are consistent with the ideal registration functionality: Sim generates the required key pairs and simulates the CA interaction consistent with $\mathcal{F}_{\text{CA}}$. Since the simulated registration transcript with the `Provider` and the certificates are distributed identically to those in the hybrid execution, no environment can distinguish the ideal execution from the hybrid execution.

Lemma 5.3. *The A-session- agent discovery and authorization protocol* $\pi_{A\text{-auth}}$ *of* $\pi_{MAGIQ}^{\mathcal{F}_{\text{scs}},\mathcal{F}_{\text{CA}},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ *described in Section 4.4 UC-realizes the A-session-authorization ideal functionality* $\mathcal{F}_{A\text{-auth}}$ *against static adversaries.*

Proof sketch. The simulator Sim runs the registration simulator described in previous Lemma, maintaining a consistent registration state. If A_I and the `Provider` are honest, the authorization request and response are generated honestly and consistently with $\mathcal{F}_{A\text{-auth}}$. If A_I is corrupted, Sim receives the request that $\mathcal{A}$ causes A_I to send, checks it against the registration state and authorization table, and returns the corresponding provider response or rejection.

Lemma 5.4. *The A-session protocol* π_{Ases} *of* $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}},\mathcal{F}_{CA},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ *(described in Section 4.4) UC realizes the A-session ideal functionality* $\mathcal{F}_{\text{Ases}}$ *against static adversaries.*

Proof sketch. Sim uses the registration and authorization simulators to maintain consistent view for registration and authorization tokens and policies, and simulates the A-session handshake and message transmission. If both agents are honest, Sim relays the session messages to $\mathcal{F}_{\text{Ases}}$ and delivers the outputs prescribed by $\mathcal{F}_{\text{Ases}}$ that enforces policies. If one agent is corrupted, Sim obtains adversarially chosen messages of that agent through $\mathcal{A}$, checks the authorization token, user and agent signatures, HMAC tags, hash-chain token progression using $\mathcal{G}_h$, and the message/time budgets using $\mathcal{G}_{\text{clk}}$. Invalid messages cause Sim sends abort to $\mathcal{F}_{\text{Ases}}$. *In-distinguishability* follows from the EUF-CMA security of the signature scheme against quantum polynomial-time adversaries, and the seudorandomness/unforgeability provided by the HMAC, PRF, and hash-chain checks in the $\mathcal{G}_h$ model.

Lemma 5.5. *The C-session protocol π_{Cses} of $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}},\mathcal{F}_{CA},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ (described in Section 4.4) UC realizes the C-session ideal functionality $\mathcal{F}_{\text{Cses}}$ against static adversaries assuming that A-session π_{Ases} UC realizes $\mathcal{F}_{\text{Ases}}$.*

Proof sketch. We consider three cases: (i) all agents are honest, (ii) only A_I^* is corrupted, and (iii) all A_R's are corrupted. In all cases Sim simulates each $\mathcal{F}_{\text{A-auth}}$ and $\mathcal{F}_{\text{Ases}}$. In case (ii), Sim receives the subtasks for each $\mathcal{F}_{\text{Ases}}$ from the corrupted A_I^* and simulates $\mathcal{F}_{\text{Ases}}$ accordingly, but in case (i) and (iii), Sim receives the task from the dummy A_I^* and it runs the task internally and gets the subtasks for each $\mathcal{F}_{\text{Ases}}$ that it will simulate.

6 Experimental Results

In this section we present experimental results. We measure the overhead of MAGIQ across cryptographic operations, protocol coordination, network communication, and bandwidth. We further evaluate MAGIQ in a multi-agent setting where an initiating agent coordinates with multiple receiving agents to complete a task.

6.1 Setup

Libraries. We use XMSS_SHA2_16_256 [31], an instance of XMSS using SHA-256, a Winternitz parameter of 16, and a 256-bit security level, to instantiate long-term agent identity keys. All inter-agent and agent–`Provider` communication is secured via post-quantum mutually authenticated TLS (PQ-mTLS), configured with X25519 and ML-KEM-768 for hybrid key exchange, ML-DSA-65 for transport-layer certificates and mutual authentication, and TLS_AES_256_GCM_SHA384 as the cipher suite. The PQ-mTLS stack is implemented using OpenSSL 3.5, while post-quantum cryptographic primitives are provided via the liboqs-python library. SHA-256 serves as the hash function across all protocol operations.

Agents. The LLM agents were implemented using AutoGen [54]. For comparison with the experimental evaluation of SAGA, we used gpt-4.1, gpt-4.1-mini, and locally hosted Qwen2.5-72B. In addition, we evaluated other cloud-hosted OpenAI models such as gpt-5.4, gpt-5.4-mini and locally deployed Qwen3.5-30B-Instruct to assess performance across a broader range of configurations.

Device Specifications. Experiments were conducted on Ubuntu 24.04, running on an AMD Ryzen Threadripper PRO 5955WX with 16 cores and 256 GB of RAM, with all reported results averaged over 100 runs. The Qwen 30B Instruct model ran on an NVIDIA RTX 4090 while Qwen-2.5 72B model used an NVIDIA H100. For all experiments, the user and the agents they own were assumed to reside on the same device.

Baseline. We use SAGA as the baseline protocol. All cryptographic operations in SAGA are built on Curve25519, with both long-term and ephemeral keys generated via the X25519 ECDH scheme using 256-bit shared secrets. Certificates follow the X.509 PKI standard and are issued by an internal certificate authority deployed as part of the `Provider`, with SHA-256 used for all digital signatures and key derivation. Cryptographic primitives were implemented using the cryptography library in Python 3.12. We used the LLM agents that came with the SAGA distribution, implemented using the `smolagents` library. Experiments were conducted with a local Qwen-2.5 72B model on an NVIDIA H100, as well as cloud-hosted OpenAI models accessed via API. The code for SAGA was obtained from the github repo [26].

6.2 Computation Overhead

Table 1 reports the computational overhead of protocol operations in MAGIQ against the SAGA baseline. Note that for SAGA, user registration, agent registration, and the setup phases on both the initiating and receiving agents are analogous to the respective handshake phases in MAGIQ. Notably, SAGA does not include counterparts to the Handshake Phase (User) or Contact Resolution (Initiating) phase, as user involvement is not required during the handshake, and contact resolution is subsumed within the setup phase of the initiating agent. As reflected in Table 1, the token decryption and validation for SAGA are incorporated into the per-interaction data transfer costs on the initiating agent, with token validation similarly accounted for on the receiving agent. Token generation by the initiating agent is likewise subsumed within its handshake phase, rather than treated as a separate operation.

On the user side, both user and agent registration in MAGIQ incur ~36 seconds, attributable to XMSS key tree pre-computation. This cost is paid only once at deployment. `Provider`-side registration, by contrast, takes at most 5.4 ms, confirming that post-quantum key management imposes no scalability burden on the infrastructure side.

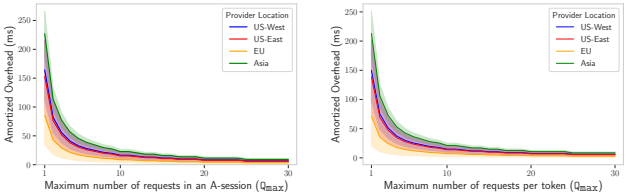
Recurring per-session costs in MAGIQ remain uniformly sub-5 ms across all operations. Contact resolution, handshake phases across all three roles, and data transfer together constitute a modest operational footprint. Data transfer in MAGIQ is effectively free at 0.03 ms per interaction, compared to 1.44 ms on the initiating side in SAGA— a reduction of nearly two orders of magnitude.

6.3 Protocol Overhead

We evaluate the overhead introduced by the access control and `Provider` coordination mechanisms of MAGIQ compared with SAGA. We consider a scenario in which an initiating agent A_I issues m requests to a receiving agent A_R, subject to a maximum of $Q_{\max}$ requests per A-sessionbetween A_I and A_R. The total overhead comprises both a network component, associated with establishing secure communication, and a cryptographic component, denoted t_{crypto}, which captures the costs of all phases involved in agent communication and data transfer, as detailed in Table 1. The total

Table 1: Computational overhead of MAGIQ and SAGA.

Operation	MAGIQ (ms)	SAGA (ms)
User Registration		
User Registration (User)	36,215.33	2.34
User Registration (Provider)	0.26	194.09
Agent Registration		
Agent Registration (User)	36,002.27	15.09
Agent Registration (Provider)	5.4	212.85
Agent Communication (per session)		
Contact Resolution (Provider)	2.96	1.46
Contact Resolution (Initiating)	4.1	N/A
Handshake Phase (Initiating)	4.01	3.17
Handshake Phase (User)	1.55	N/A
Handshake Phase (Receiving)	4.71	1.83
Data Transfer (per interaction)		
Initiating	0.03	1.44
Receiving	0.03	0.26



(a) MAGIQ **(b) SAGA**

Figure 7: Amortized protocol overhead across provider locations (US-West, US-East, EU, Asia) under varying $Q_{\max}$ for $m = 100$ requests between A_I and A_R. Shaded regions reflect the variability in overhead attributable to differences in agent location worldwide.

protocol overhead is modeled as:

$$c_{\text{proto}}(m) = \left(\text{RTT}_{A_i,P} + t_{\text{crypto}}\right) \cdot \left\lceil \frac{m}{Q_{\max}} \right\rceil, \tag{1}$$

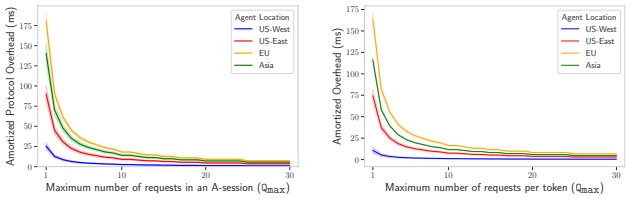
where P is the `Provider` and $\text{RTT}_{A_i,P}$ is the round-trip time between A_I and the `Provider`. $t_{\text{crypto}} = 20.33$ ms is the measured cryptographic overhead per A-session. Each authorization cycle must be performed once every $Q_{\max}$ requests.

We sample round-trip times from empirical measurement distributions using monitors in US-East, US-West, Europe, and Asia, made available by CAIDA [8] and AWS [1]. Figures 7 and 8 show the amortized protocol overhead:

$$\bar{c}_{\text{proto}}(m) = \frac{c_{\text{proto}}(m)}{m} \tag{2}$$

as a function of $Q_{\max}$, using $m = 100$ requests, under two configurations: one in which the `Provider` location varies across four geographic regions while agents are distributed worldwide (Fig. 7), and one in which the `Provider` is fixed in US-West while the initiating agent A_I is placed across the same four regions (Fig. 8).

Across both configurations, MAGIQ and SAGA exhibit qualitatively identical amortization behavior: overhead decreases sharply



(a) MAGIQ **(b) SAGA**

Figure 8: Amortised protocol overhead across A_I locations (US-West, US-East, EU, Asia) under varying $Q_{\max}$ for $m = 100$ requests between A_I and A_R. Shaded regions reflect the variability in overhead attributable to differences in network conditions across agent locations worldwide.

Table 2: Comparison of task execution time (in seconds) between SAGA and MAGIQ. We assume that A_i, A_r, and the `Provider` are located in Europe, Asia and US-West respectively, with $Q_{\max} = 10$ requests per session.

			Networking (s)		Overhead (s)	
Task	**LLM Backend**	**LLM (s)**	SAGA	MAGIQ	SAGA	MAGIQ
Calendar	GPT-4.1-mini	35.524	0.791	1.140	0.165	0.176
Email	GPT-4.1	17.8	1.319	1.140	0.165	0.176
Writing	Qwen-2.5	N/A	1.319	N/A	0.165	N/A

with increasing $Q_{\max}$ in all regional configurations, falling below 40 ms by $Q_{\max} = 10$ and converging to within 10 ms of one another by $Q_{\max} = 15$. Although MAGIQ incurs an additional post-quantum cryptographic overhead absent in SAGA, this cost is subsumed by the latency variation attributable to geographic separation, which constitutes the dominant source of variability in both systems. Consequently, the two systems are indistinguishable in their convergence characteristics at practical session sizes: an Asia-based agent at $Q_{\max} = 15$ incurs overhead comparable to a co-located deployment at $Q_{\max} = 5$, regardless of whether the `Provider` or the agent location is varied, confirming that a modest increase in session capacity effectively neutralizes geographic latency disparity and that the marginal cost of post-quantum security in MAGIQ is not discernible under realistic network conditions.

6.4 Task Completion Overhead

We evaluate task execution overhead for SAGA and MAGIQ across the three agentic tasks from SAGA: Calendar, Email, and Writing, adopting the same deployment configuration—initiating agent A_I in Europe, receiving agent A_R in Asia, `Provider` in US-West, and $Q_{\max} = 10$ requests per A-session in between A_I and A_R.

Table 2 presents a direct comparison between SAGA and MAGIQ. The measured overhead of SAGA is 0.165 s, while MAGIQ incurs 0.176 s, corresponding to a modest 1.07× increase due to post-quantum cryptographic operations. The protocol overhead is dominated by a single `Provider` round-trip per $Q_{\max}$ requests (Equation 1). Nevertheless, in both systems the protocol overhead constitutes a negligible fraction of cumulative LLM inference time. For

Table 3: Task execution time (in seconds) for MAGIQ. We assume that A_I, A_R, and `Provider` are located in Europe, Asia, and US-West respectively, with $Q_{\max} = 10$ requests per session.

Task	LLM Backend	LLM (s)	Network (s)	MAGIQ Overhead (s)
Calendar	GPT-5.4	12.507	0.899	0.176
	GPT-5.4-mini	11.367	0.899	
	Qwen-3.5	7.275	1.140	
Email	GPT-5.4	14.009	0.656	0.176
	GPT-5.4-mini	8.848	0.656	
	Qwen-3.5	7.298	0.899	
Writing	GPT-5.4	30.498	1.140	0.176
	GPT-5.4-mini	10.421	0.899	
	Qwen-3.5	10.652	1.140	

the Calendar and Email tasks, the MAGIQ overhead of 0.176 s represents less than 0.8% of cumulative LLM inference time in both cases. The Writing task could not be completed under Qwen-2.5, as the model's tool-call outputs did not conform to the format expected by the agent framework; we attribute this to the limitations of an older model generation.

Table 3 evaluates MAGIQ across the same three tasks using current-generation LLMs: GPT-5.4, GPT-5.4-mini, and Qwen-3.5. Across all configurations, MAGIQ overhead remains fixed at 0.176 s, confirming that it is independent of the underlying LLM, as expected from the protocol design. The newer models yield substantially reduced LLM inference times and varying networking costs, reflecting the differing number of inter-agent interactions required to complete each task. Qwen-3.5 achieves the lowest inference times on Calendar (7.275 s) and Email (7.298 s) tasks, while GPT-5.4-mini offers competitive performance across all three tasks at reduced cost relative to GPT-5.4. In all cases, MAGIQ overhead of 0.176 s represents well under 2% of total execution time, confirming that the cost of post-quantum access control is effectively subsumed by LLM inference latency across all evaluated models and task types.

6.5 Performance Evaluation of Provider

We evaluate the cumulative overhead imposed on the `Provider` as a function of A-session lifetime, varying the number of initiating agents. Because each new A-session requires one contact-resolution operation at the `Provider` (2.96 ms, Table 1), a shorter session lifetime increases the rate at which this operation must be performed and thus the total `Provider` burden. For n initiating agents each refreshing sessions of lifetime ℓ minutes, the daily overhead is:

$$C_{\texttt{Provider}}(n, \ell) = n \cdot \frac{1440}{\ell} \cdot t_{\text{crypto}}, \tag{3}$$

where $t_{\text{crypto}} = 2.96$ ms is the measured contact-resolution cost and 1440 is the number of minutes in a day. Compared with SAGA, which incurs a contact-resolution cost of 1.46 ms per-session, MAGIQ introduces approximately 2× the per-session `Provider` cost, owing to its additional post-quantum cryptographic operations.

Figure 9 plots $C_{\texttt{Provider}}$ for $n \in \{1, 10, 100\}$ agents across session lifetimes ranging from one minute to one day. Two structural observations follow directly from Equation 3. First, overhead scales linearly with agent count: the 100-agent curve lies exactly one order of magnitude above the 10-agent curve at every point. Second, overhead is inversely proportional to session lifetime: moving from a 1-minute to a 1-day session reduces `Provider` cost by three orders of magnitude for any fixed agent population. Even accounting for the 2× increase in per-session cost relative to SAGA, the absolute `Provider` burden remains negligible across all evaluated configurations, confirming that the post-quantum overhead of MAGIQ does not materially alter the scalability characteristics of the system.

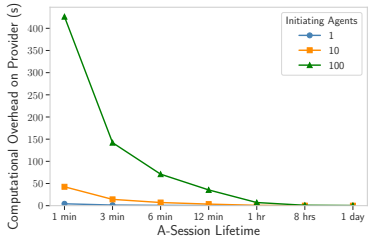


Figure 9: Computational overhead on the `Provider` as a function of A-sessionlifetime, for 1, 10, and 100 initiating agents.

Table 4: Bandwidth requirements (in KB) per protocol phase for SAGA and MAGIQ. SAGA parameters: $N = 100$ OTKs, $m = 10$, $Q_{\max} = 10$. MAGIQ parameters: $m = 10$, $Q_{\max} = 10$.

Operation	SAGA	MAGIQ
User Registration	0.6	6.98
Agent Registration	10.40	27.28
Agent Communication (per session)	2.12	88.65
Data Transfer (per request)	0.59	0.56

6.6 Bandwidth Overhead

Table 4 summarizes the bandwidth requirements across protocol phases for SAGA and MAGIQ. For SAGA, we assume $N = 100$ one-time keys (OTKs) are registered by the user on behalf of the agent. For both protocols, we consider a setting in which an initiating agent A_I issues $m = 10$ requests to a receiving agent A_R, with a maximum of $Q_{\max} = 10$ requests per A-session.

User and agent registration incur one-time costs and therefore do not affect per-task bandwidth. In SAGA, these costs amount to 0.6 KB for user registration and 10.40 KB for agent registration, whereas in MAGIQ they increase to 6.98 KB and 27.28 KB, respectively. The most significant difference arises during session establishment, which constitutes the recurring per-token-cycle overhead: SAGA requires 2.12 KB per cycle, while MAGIQ incurs 88.65 KB, representing a ∼42× increase. The bulk of this overhead is due to the post quantum certificates and signatures being transmitted in between A_I and A_R during the handshake phase. In contrast, per-request data transfer remains comparable between the two protocols, at 0.59 KB for SAGA and 0.56 KB for MAGIQ. When using MAGIQ for tasks involving more than 2 agents, only the bandwidth required for the agent communication phase grows linearly with the number of agents. Specifically, with t agents the cost scales $(t-1)\times$ the per session bandwidth.

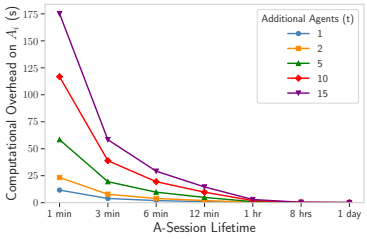


Figure 10: Daily computational overhead on A_I as a function of A-session lifetime, for $t \in \{1, 2, 5, 10, 15\}$ receiving agents. Per-session cost follows Equation 4 using measured cryptographic costs; daily overhead follows Equation 5. All A-sessions are assumed to share the same lifetime.

6.7 One-Agent to Many-Agents Overhead

We evaluate the overhead borne by an *orchestrator* (A_I) that coordinates with t receiving agents to complete a task. For each receiving agent A_R, A_I must establish one A-session comprising a contact-resolution step and a handshake phase. We assume all A-sessions share the same lifetime ℓ and that each A-session comprises Q_{max}=10 requests. The per-session cryptographic cost on A_I is:

$$c_{\text{init}}(t) = \underbrace{4.1\,t}_{\text{contact resolution}} + \underbrace{4.01\,t}_{\text{handshake}} = 8.11\,t \quad \text{(ms)}, \tag{4}$$

where costs are taken from Table 1. Because a new session must be established every ℓ minutes, the total daily overhead on A_I is:

$$C_{\text{init}}(t,\ \ell) = c_{\text{init}}(t) \cdot \frac{1440}{\ell}, \tag{5}$$

with 1440 denoting the number of minutes in a day. Figure 10 plots C_{init} for $t \in \{1, 2, 5, 10, 15\}$ across session lifetimes ranging from one minute to one day. The overhead is governed by two independent axes. At a fixed lifetime, cost scales linearly with t: at a 1-minute session lifetime, overhead ranges from ∼12 seconds for $t = 1$ to ∼175 seconds for $t = 15$, a 15× ratio consistent with Equation 4. At a fixed t, cost falls inversely with ℓ: for $t = 15$, extending the session lifetime from 1 minute to 1 hour reduces daily overhead from ∼175 seconds to under 3 seconds, and to under 0.4 seconds at 8 hours or beyond. For all values of t, the curves converge to negligible overhead beyond a 1-hour lifetime, confirming that session lifetime is the dominant design parameter and that MAGIQ imposes no meaningful burden on A_I under practical deployment conditions.

6.8 Multi-Agent Task Completion Overhead

We evaluate task execution overhead for MAGIQ across the three agentic tasks—Calendar, Email, and Writing—extending the two-agent deployment of MAGIQ to a multi-agent setting. For both MAGIQ and SAGA the *orchestrator* (A_I) is in Europe, receiving agents A_R in Asia, and the Provider in US-West, with $Q_{\max} = 10$ requests per A-session between A_I and A_R. Each task is remodeled for the multi-agent setting, comprising one *orchestrator* agent and two receiving agents executing a sequential, static workflow, where the total overhead encompasses task completion across all three agents. As reported in Table 5, MAGIQ incurs a constant overhead of 0.336 s across all tasks and LLM backends—identical to that observed in the two-agent case (Table 4, 0.176 s)—confirming that the protocol cost scales predictably with the number of agents. Crucially, this overhead remains well under 4% of the LLM inference time across all configurations (ranging from 10.421 s to 66.731 s), demonstrating that MAGIQ's overhead is effectively subsumed by model inference latency, irrespective of agent topology or task type.

Table 5: Task execution time (seconds) for MAGIQ with 3 agents. A_I is located in Europe, 2 A_R-s in Asia, and `Provider` in US-West, with $Q_{\max} = 10$ requests per session.

Task	LLM	LLM(s)	Network(s)	MAGIQ (s)
Calendar	GPT-5.4	50.606	1.708	0.336
	GPT-5.4-mini	42.203	1.950	
	Qwen-3.5	23.487	1.950	
Email	GPT-5.4	47.669	1.223	0.336
	GPT-5.4-mini	34.261	1.465	
	Qwen-3.5	31.194	1.708	
Writing	GPT-5.4	66.731	1.223	0.336
	GPT-5.4-mini	10.421	1.223	
	Qwen-3.5	36.471	2.192	

7 Related Work

Designs for Agent Governance. Recent work has moved beyond static permissions toward dynamic, machine-verifiable delegation. Chen [17] introduces AITH, a post-quantum continuous delegation protocol that replaces discrete signing events with sub-microsecond boundary checks, allowing agents to operate autonomously within cryptographically attested "standing instructions". Addressing the non-deterministic nature of agent behavior, Bhardwaj [5] proposes Agent Behavioral Contracts (ABC), which apply Design-by-Contract principles to enforce preconditions, invariants, and recovery mechanisms at runtime. Kaptein et al. [34] further formalize this via "Policies on Paths," arguing that governance must treat the agent's execution trajectory as a deterministic function of its identity and prior actions to prevent information barrier breaches.

Inter-Agent Protocol Implementations. The transition toward an "Agentic Web" has spurred the development of standardized communication layers. While early implementations like Agent Protocol from langchain lacked robust security, the Agent Communication Protocol (ACP) [35] now provides a federated orchestration model integrating decentralized identifiers (DIDs) and automated service-level agreements. Google's Agent-to-Agent (A2A) protocol [51] facilitates discovery via "Agent Cards," though it lacks the centralized mediation found in the SAGA architecture. Furthermore, Anbiaee et al. [3] provide a systematic security analysis of emerging protocols (MCP, A2A, Agora, and ANP), identifying design-induced risk surfaces such as token scope escalation and naming collisions.

Attacks on Multi-Agent Systems. Several works examine adversarial propagation in multi-agent communication [2, 27, 28, 37, 55], where rogue agents can propagate malicious outputs via interactions with other agents. Triedman et al. [33] demonstrate Control-Flow Hijacking (CFH) attacks, where orchestrators are manipulated via indirect prompt injection to misroute sub-tasks to malicious agents. Chen et al. [16] introduce AdapAM, a framework for black-box adversarial attacks that utilizes adaptive selection to induce malicious actions in victim agents with high stealth. Wang et al.

[53] introduces the "Kill-Chain Canary" methodology, which tracks cryptographic tokens through discrete stages (Exposed, Persisted, Relayed, Executed) to localize where defenses fail.

8 Conclusion

In this paper we presented MAGIQ, a framework for policy definition and enforcement of multi-agentic AI systems using novel highly efficient quantum-resistant cryptographic protocols with proved security guarantees. MAGIQ (i) allows users to define rich communication and access control policy budgets; (ii) enforces such policies using post-quantum cryptographic primitives; (iii) supports session-based enforcement of policies for agent-to-agent and one-to-many agent sessions; and (iv) provides accountability of agents to their users in the form of *task-msg* attribution. We formally model and prove correctness and security of the system using the UC framework. Our evaluation results demonstrate that MAGIQ is a viable solution for agentic AI governance systems.

References

[1] Matt Adorjan. 2025. cloudping.co: AWS Inter-Region Latency Monitoring. https://github.com/mda590/cloudping.co Accessed: 2025-04-18.

[2] Alfonso Amayuelas, Xianjun Yang, Antonis Antoniades, Wenyue Hua, Liangming Pan, and William Yang Wang. 2024. MultiAgent Collaboration Attack: Investigating Adversarial Attacks in Large Language Model Collaborations via Debate. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 6929–6948.

[3] Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. 2026. Security Threat Modeling for Emerging AI-Agent Protocols: A Comparative Analysis of MCP, A2A, Agora, and ANP. arXiv:2602.11327 [cs.CR] https://arxiv.org/abs/2602.11327

[4] Sepideh Avizheh, Mahmudun Nabi, and Reihaneh Safavi-Naini. 2024. Refereed delegation of computation using smart contracts. *IEEE Transactions on Dependable and Secure Computing* 21, 6 (2024), 5208–5227.

[5] Varun Pratap Bhardwaj. 2026. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. doi:10.5281/ZENODO.18775393

[6] Johannes Buchmann, Erik Dahmen, Sarah Ereth, Andreas Hülsing, and Markus Rückert. 2013. On the security of the Winternitz one-time signature scheme. *International Journal of Applied Cryptography* 3, 1 (2013), 84–96.

[7] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. 2011. XMSS-a practical forward secure signature scheme based on minimal security assumptions. In *International Workshop on Post-Quantum Cryptography*. Springer, 117–129.

[8] CAIDA. [n. d.]. The CAIDA Archipelago Monitor Statistics. https://www.caida.org/projects/ark/statistics/. Accessed April 2025.

[9] Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. 2018. The wonderful world of global random oracles. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 280–312.

[10] Ran Canetti. 2001. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. IEEE, 136–145.

[11] Ran Canetti. 2004. Universally composable signature, certification, and authentication. In *Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004.* IEEE, 219–233.

[12] Ran Canetti, Kyle Hogan, Aanchal Malhotra, and Mayank Varia. 2017. A universally composable treatment of network time. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 360–375.

[13] Ran Canetti, Pratik Sarkar, and Xiao Wang. 2020. Efficient and round-optimal oblivious transfer and commitment with adaptive security. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 277–308.

[14] Alan Chan, Noam Kolt, Peter Wills, Usman Anwar, Christian Schroeder de Witt, Nitarshan Rajkumar, Lewis Hammond, David Krueger, Lennart Heim, and Markus Anderljung. 2024. IDs for AI Systems. *arXiv preprint arXiv:2406.12137* (2024).

[15] Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elija Perrier, Seth Lazar, Gillian K. Hadfield, and Markus Anderljung. 2025. Infrastructure for AI Agents. *arXiv preprint arXiv:2501.10114* (2025).

[16] Jianming Chen, Yawen Wang, Junjie Wang, Xiaofei Xie, Yuanzhe Hu, Qing Wang, and Fanjiang Xu. 2026. Adversarial Attack on Black-Box Multi-Agent by Adaptive Perturbation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 35 (Mar. 2026), 29359–29367. doi:10.1609/aaai.v40i35.40176

[17] Zhaoliang Chen. 2026. AITH: A Post-Quantum Continuous Delegation Protocol for Human-AI Trust Establishment. arXiv:2604.07695 [cs.CR] https://arxiv.org/abs/2604.07695

[18] Model Context Protocol Contributors. 2025. Model Context Protocol Registry. https://github.com/modelcontextprotocol/registry. Accessed: 2025-12-11.

[19] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. https://arxiv.org/abs/2503.18813

[20] Stefan Dziembowski, Lisa Eckey, and Sebastian Faust. 2018. Fairswap: How to fairly exchange digital goods. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 967–984.

[21] Lisa Eckey, Sebastian Faust, and Benjamin Schlosser. 2020. Optiswap: Fast optimistic fair exchange. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*. 543–557.

[22] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. 2020. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU, Specification v1.2.* Cryptographic Specification. falcon-sign.info. https://falcon-sign.info/falcon.pdf Accessed: 2026-02-12.

[23] Sebastian Gajek, Mark Manulis, Olivier Pereira, Ahmad-Reza Sadeghi, and Jörg Schwenk. 2008. Universally composable security analysis of TLS. In *International Conference on Provable Security*. Springer, 313–327.

[24] Google Developer Blog. 2025. Announcing the Agent2Agent Protocol (A2A). https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/. Accessed: 2025-07-22.

[25] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (Philadelphia, Pennsylvania, USA) *(STOC '96)*. Association for Computing Machinery, New York, NY, USA, 212–219. doi:10.1145/237814.237866

[26] gsiros. 2024. saga. https://github.com/gsiros/saga

[27] Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast.

[28] Pengfei He, Yupin Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. 2025. Red-teaming llm multi-agent systems via communication attacks. *arXiv preprint arXiv:2502.14847* (2025).

[29] Julia Hesse, Stanislaw Jarecki, Hugo Krawczyk, and Christopher Wood. 2023. Password-authenticated TLS via OPAQUE and post-handshake authentication. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 98–127.

[30] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VtmBAGCN7o

[31] Andreas Huelsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. 2018. XMSS: eXtended Merkle Signature Scheme. RFC 8391. doi:10.17487/RFC8391

[32] Andreas Hülsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. 2018. XMSS: eXtended Merkle Signature Scheme. RFC 8391. doi:10.17487/RFC8391

[33] Rishi Jha, Harold Triedman, Justin Wagle, and Vitaly Shmatikov. 2026. Breaking and Fixing Defenses Against Control-Flow Hijacking in Multi-Agent Systems. arXiv:2510.17276 [cs.LG] https://arxiv.org/abs/2510.17276

[34] Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. 2026. Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586 [cs.AI] https://arxiv.org/abs/2603.16586

[35] Naveen Kumar Krishnan. 2026. Beyond Context Sharing: A Unified Agent Communication Protocol (ACP) for Secure, Federated, and Autonomous Agent-to-Agent (A2A) Orchestration. arXiv:2602.15055 [cs.MA] https://arxiv.org/abs/2602.15055

[36] Leslie Lamport. 1979. Constructing digital signatures from a one way function. *Technical Report SRI-CSL-98* (1979).

[37] Donghyun Lee and Mo Tiwari. 2024. Prompt infection: Llm-to-llm prompt injection within multi-agent systems. *arXiv preprint arXiv:2410.07283* (2024).

[38] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. 2026. ACE: A Security Architecture for LLM-Integrated App Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

[39] Yedidel Louck, Ariel Stulman, and Amit Dvir. 2025. Improving Google A2A Protocol: Protecting Sensitive Data and Mitigating Unintended Harms in Multi-Agent Systems. arXiv:2505.12490 [cs.CR] https://arxiv.org/abs/2505.12490

[40] Dustin Moody, Ray Perlner, Andrew Regenscheid, Angela Robinson, and David Cooper. 2024. *Transition to Post-Quantum Cryptography Standards*. Technical

Report NIST IR 8547 (Initial Public Draft). National Institute of Standards and Technology, Gaithersburg, MD, USA. doi:10.6028/NIST.IR.8547.ipd Initial Public Draft.
[41] Luca Muscariello, Vijoy Pandey, and Ramiz Polic. 2025. The AGNTCY Agent Directory Service: Architecture and Implementation. arXiv:2509.18787 [cs.AI] https://arxiv.org/abs/2509.18787
[42] National Institute of Standards and Technology (NIST). August 13, 2024. FIPS 203 Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final Available at https://csrc.nist.gov/pubs/fips/203/final.
[43] National Institute of Standards and Technology (NIST). August 13, 2024. FIPS 204 Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final Available at https://csrc.nist.gov/pubs/fips/204/final.
[44] National Institute of Standards and Technology (NIST). August 13, 2024. FIPS 205 Stateless Hash-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/205/final Available at https://csrc.nist.gov/pubs/fips/205/final.
[45] Ramesh Raskar, Pradyumna Chari, Jared James Grogan, Mahesh Lambe, Robert Lincourt, Raghu Bala, Aditi Joshi, Abhishek Singh, Ayush Chopra, Rajesh Ranjan, Shailja Gupta, Dimitris Stripelis, Maria Gorskikh, and Sichao Wang. 2025. Upgrade or Switch: Do We Need a Next-Gen Trusted Architecture for the Internet of AI Agents? arXiv:2506.12003 [cs.NI] https://arxiv.org/abs/2506.12003
[46] Tirumaleswar Reddy and Hannes Tschofenig. 2025. Post-Quantum Cryptography Recommendations for TLS-based Applications. Internet-Draft, draft-ietf-uta-pqc-app-00. https://www.ietf.org/archive/id/draft-ietf-uta-pqc-app-00.html Work in progress.
[47] Ronald L Rivest and Adi Shamir. 1996. PayWord and MicroMint: Two simple micropayment schemes. In *International workshop on security protocols*. Springer, 69–87.
[48] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, et al. 2023. Practices for governing agentic AI systems. *Research Paper, OpenAI* (2023).
[49] P.W. Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 124–134. doi:10.1109/SFCS.1994.365700
[50] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. 2025. Authenticated Delegation and Authorized AI Agents. *arXiv preprint arXiv:2501.09674* (2025).
[51] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. Announcing the Agent2Agent Protocol (A2A). Google Developers Blog. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ Accessed: 2025-04-10.
[52] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2026. SAGA: A Security Architecture for Governing AI Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
[53] Haochuan Kevin Wang and Zechen Zhang. 2026. Kill-Chain Canaries: Stage-Level Tracking of Prompt Injection Across Attack Surfaces and Model Safety Tiers. arXiv:2603.28013 [cs.CR] https://arxiv.org/abs/2603.28013
[54] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155
[55] Weichen Yu, Kai Hu, Tianyu Pang, Chao Du, Min Lin, and Matt Fredrikson. 2025. Infecting LLM Agents via Generalizable Adversarial Attack. In *Red Teaming GenAI: What Can We Learn from Adversaries?* https://openreview.net/forum?id=udsmFGMwlp
[56] Weibo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. 2025. When MCP Servers Attack: Taxonomy, Feasibility, and Mitigation. arXiv:2509.24272 [cs.CR] https://arxiv.org/abs/2509.24272

A Ethical Considerations

Our paper is not an attack paper, it does not use any public dataset, or human data collection, so we believe that there are no ethical concerns.

B Notations

We present the notations used throughout the paper in Table 6.

C Cryptographic Enforcement of Policies

Below we define each of the three policies of CP, RCP, and ICP and give the mechanisms that are used to enforce each of them in our protocol.

Table 6: Notations

Symbol	Description
Entities	
A_I	Initiator agent
A_R	Responding agent
A_I^*	Orchestrator agent
U	User
TA	Provider
CA	Certificate authority
MAGIQ Protocols	
$Cert$	Certificate issued by CA
uid_U	User U identifier
aid_A	Agent A identifier
ED_A	Agent A endpoint descriptor
M_A	Metadata of agent A
(pk_A^{tls}, sk_A^{tls})	Agent A public and private PQ-TLS credentials
(pk_A, sk_A)	Agent A public and private keys for hash based signature
CP_A	Contact policy of Agent A (ACL and A-session budget)
σ_X^Y	Signature generated by Y with type X
$NextTok$	Next message count token for *task-msg* budget
DU	Provider's user registry
DA	Provider's agent registry
HS	Hash-based signature
req	Request
res	Response
s_0, ρ_0	Random seeds for *task-msg* count tokens
$Mroot$	Merkle root
n, n'	hash chain length
Security Model and Analysis	
Sim	Simulator
Sim^A	Simulator with oracle access to A
$\mathcal{A}$	Adversary
$\mathcal{Z}$	Environment
$\mathcal{F}$	Ideal functionality
$\mathcal{G}$	Global ideal functionality
π	Protocol
$\pi^{\mathcal{F}_1,\mathcal{F}_2,\cdots}$	Hybrid protocol that uses $\mathcal{F}_1, \mathcal{F}_2, \cdots$ as subroutines
τ	Current time read from $\mathcal{G}_{\mathsf{clk}}$
T_{exp}	Expiry time
Q	*task-msg* budget
Δ	Time budget
sid	Session identifier
IDI	Initiator's identity (in $\mathcal{F}_{\mathsf{SCS}}$)
P, P'	Party (in $\mathcal{F}_{\mathsf{SCS}}$)
I	Initiator
R	Responser
L	Set representation
$Apolicy$	Policy of agent consisting of *task-msg* and time budgets
$\ell()$	Leakage function
ℓ	Leakage value
ctr	Counter
m	*task-msg*
$attr_A$	Attribute of agent A
Pwd	Password
$tsk, subtsk$	Task and subtask, respectively
$Budget$	Contact budget
$ExeReq()$	The request execution function
$ExeRes()$	The response execution function

C.1 Contact Policy *CP*

Users define and administer contact policies for their registered agents. The contact policy *CP* states that which agents can establish a connection with whom and for how many times (budget). *CP* consists of a set of declarative rules such as ("receive, *@company.com:email-agent", 10) which allows an email handling agent from a specified domain to initiate contact at most 10 times, or ("send, *@company.com:email-agent", 20) which allows the agent to initiate an A-session with an email handling agent for 20 times. If multiple rules match, the one with the highest specificity is selected. The number of connections between an initiating agent A_I and responding agent A_R, i.e., the budget, is defined as below:

$$Budget(aid_{A_R}, aid_{A_I}) = \begin{cases} -1 & \text{if } R = \emptyset \\ B(r^*) & \text{if } R \neq \emptyset \end{cases}$$

Which state that r^* is the most specific rule among all rules $R \in CP_{A_R}$ when A_R and A_I are establishing an A-session, and $B(r^*)$ corresponds to the rule r^*. $R = \emptyset$ allows the user to distinguish between no match in policy and an expired budget.

Enforcing the contact policy *CP***:** when an initiating agent A_I queries the provider to contact A_R, the provider first verifies that the agents participating in the A-session satisfy the contact policy $CP_{A_R} \cap CP_{A_I}$, which means that $CP_{A_R} \cap CP_{A_I} \neq \emptyset$. If this is the first time A_I and A_R connects, then the provider creates a counter $Counter[aid_{A_R}][aid_{A_I}]$ to keep track of the number of contacts and initializes it with $Min\{Budget(aid_{A_R}, aid_{A_I}), Budget(aid_{A_R}, aid_{A_I})\}$.

If the policy check succeeds, the provider returns a signature σ_{ac}^{TA} to agent A_I on the recipient's data and decreases the counter by one: $\sigma_{ac}^{TA} = HS.Sign_{sk_{TA}}(\nu, Cert_{U_R}^{ID}, aid_{A_R}, M_{A_R}, \sigma_{ID}^{U_R}, \sigma_{A_R}^{U_R})$, where ν is a nonce used to ensure token freshness. We assume the receiver agent will keep track of the nonce. If *Budget* is negative, or exhausted, then provider sends a failure message and terminates the connection. The user of agent A_R can update the contact policy and dedicate a new budget at any time.

C.2 Responder Contact Policy *RCP*

We consider RCP is defined by the user for a responding agent A_R, and consists of the tuples of the form $(A_R, A_I, Q_{R,I}, \Delta_{R,I})$ which states that in an A-session between A_I and A_R, the maximum number of requests that A_R can receive is $Q_{R,I}$, and the maximum duration of the A-session is $\Delta_{R,I}$.

Enforcing the interaction budget in *RCP***:** We assume the responding agent can be corrupted and define a scheme which allows the responding agent to keep the total budget counting correctly, while it ensures the budget has indeed been determined by the user, and the initiating agent can *efficiently* verify when the responding agent exceed the maximum allowed budget.

- When A_I establishes an A-session with A_R, the responder agent A_R derives a random seed:

  $$s_0 = \mathsf{PRF}(sid,\ aid_{A_I},\ aid_{A_R},\ addr)$$

  where *addr* is the chain index for this A-session (set to 0 for the first chain). It then builds a *personalized hash chain* of length $n = Q_{R,I}$:

  $$s_j = H(s_{j-1},\ j,\ sid,\ aid_{A_I}), \qquad j = 1, \ldots, n$$

  Binding aid_{A_I} into each step prevents a corrupted A_R from reusing the chain with another agent to exceed $Q_{R,I}$; binding *sid* prevents reuse across A-sessions; and the index j allows A_I to track which hash values A_R opens and detect any point of cheating. These values serve as *hash-based message count tokens* that enforce the interaction budget. Agent A_R then sends the terminal value s_n to the user U_R.
- User U_R signs the terminal value and chain length:

  $$\sigma_{RCP}^{U_R} = \mathsf{HS.Sign}_{sk_{U_R}}(s_n,\ Q_{R,I})$$

  and sends $\sigma_{RCP}^{U_R}$ to A_R.
- Agent A_R forwards s_n, s_{n-1}, $Q_{R,I}$, $\Delta_{R,I}$, and $\sigma_{RCP}^{U_R}$ to the initiator A_I.
- Agent A_I verifies the user's signature:

  $$\mathsf{HS.Verify}_{pk_{U_R}}\left(\langle\, s_n,\ Q_{R,I}\,\rangle,\ \sigma_{RCP}^{U_R}\right) = 1$$

  and checks that $Q_{R,I} = n$ and $H(s_{n-1},\ n,\ sid,\ aid_{A_I}) = s_n$. If all checks pass, A_I is assured that A_R has correctly initialized the interaction budget, and initializes a counter $ctr_{RCP} = Q_{R,I}$, decrementing it by one. In subsequent rounds, each time A_I receives a token it verifies:

  $$s_i = H(s_{i-1},\ i,\ sid,\ aid_{A_I})$$

  and decrements ctr_{RCP}. When the counter reaches zero, A_I performs a final check:

  $$s_0 = \mathsf{PRF}(sid,\ aid_{A_I},\ aid_{A_R},\ addr)$$

  At this point A_R must cease accepting further requests; an honest A_I will stop sending them regardless, though a corrupted A_R may not comply.

We guarantee that if both agents that are involved in an A-session are honest or only one of them is honest then the interaction budget will be correctly enforced. Note that we do not ensure any guarantee for the case when both agents are corrupted.

C.3 Initiator Contact Policy *ICP*

We assume that the user chooses Q_{tot}, that is, the maximum number of requests for an initiator agent that can send to other agents toward fulfilling a task, and Δ_{tot} that is the maximum duration of time that an initiator agent can interact with other agents to fulfill a given task. These two values are stored as the Initiator contact policy *ICP*, that is, $ICP = (Q_{tot}, \Delta_{tot})$.

Enforcing the interaction budget in *ICP* **(static workflow).** We assume that the *orchestrator* agent A_I^* acts as an initiator and orchestrates the communication with other agents $A_1, A_2, \cdots, A_t$ through sessions $S_1, S_2, ..., S_t$. We design a scheme that enforces the total budget Q_{tot} throughout all these sessions that allows A_I^* to keep the total budget counting correctly while ensuring that the budget has indeed been determined by user U^*. Also, the responding agents $A_1, A_2, A_3, ...$, can *efficiently* identify when A_I^* exceeds the budget without knowing the history of interactions of A_I^* with other agents and without interacting with other agents (just using their local views). We require that the communication should stay almost constant (independent of the request number) for each pair of agents.

We note that a single hash chain of length Q_{tot} will not work for *ICP* since the agents need to know the history of the consumed hash tokens in order to verify the token presentations. This will also lead to a communication cost that grows linearly with the number of requests. Therefore, this naive approach will not work. To circumvent around this, we first consider that the agents follow an static workflow where the *orchestrator* agent knows which agents it has to contact to fulfill the task as soon as it receives the task. We design the hash-based tokens to enforce the interaction budget for an static workflow as below.

- The user U^* determines the maximum interaction budget Q_{tot} for *ICP*.
- Upon receiving a task from U^*, agent A_I^* processes it and obtains the workflow, identifying the list of agents $A_1, \ldots, A_t$ to contact. It then creates m personalized hash chains of length n, where each chain is seeded as:

  $$s_0 = \mathsf{PRF}(sid,\ aid_{A_I^*},\ aid_{A_i},\ addr)$$

  and extended as $s_j = H(s_{j-1},\ j,\ sid,\ aid_{A_i})$ for $j = 1, \ldots, n$. The parameters satisfy $Q_{tot} = n \times m$ and $m \geq t$, ensuring the total chain length does not exceed Q_{tot} and that at least one chain is available per A-session; A_I^* may allocate two or more chains to a single A-session. Agent A_I^* then sends the terminal values $\{s_n, s'_n, \ldots, s_n^{\ell}\}$ of all personalized hash chains to user U^*.
- The user U^* constructs a Merkle tree over the terminal values and signs the root together with the chain length:

  $$\sigma_{ICP}^{U^*} = \mathsf{HS.Sign}_{sk_{U^*}}(Mroot,\ n),$$

  and sends the Merkle tree and signature $\sigma_{ICP}^{U^*}$ to A_I^*.
- When A_I^* initiates an A-session with agent A_i, $i \in [1, t]$, it sends to A_i:
  - the Merkle root *Mroot* and user signature $\sigma_{ICP}^{U^*}$
  - a Merkle proof that the hash chain root is included in the tree signed by U^*
  - the chain openings s_n and s_{n-1}, where $s_n = H(s_{n-1},\ n,\ sid,\ aid_{A_i})$
- Agent A_i verifies the following:
  - the user's signature: $\mathsf{HS.Verify}_{pk_{U^*}}(\langle Mroot,\ n\rangle,\ \sigma_{ICP}^{U^*}) = 1$
  - the Merkle proof is valid
  - the hash step: $s_n = H(s_{n-1},\ n,\ sid,\ aid_{A_i})$

  If all checks pass, A_i is assured that U^* has authorized the use of a personalized hash chain of length n for agent A_i to fulfill the task. Agent A_i then initializes a counter $ctr_{ICP} = n$ and decrements it by one.

In the next rounds, the initiator agent just sends a hash token from the chain s_j, and the responding agent A_i verifies that $s_j = H(s_{j-1}, j, sid, aid_{A_i})$, and then decreases the counter ctr_{ICP}. When the counter ctr_{ICP} reaches 0 it checks that $s_0 = PRF(sid, aid_{A_I^*}, aid_{A_i}, addr)$. If the agent wants to continue interactions and use more hash chains, it can increment *addr* by 1, and repeat the last two steps mentioned above to start using a new personalized hash chain.

Enforcing the interaction budget in *ICP* (dynamic workflow). For a dynamic workflow, the *orchestrator* agent cannot know the list of agents who wants to contact with them ahead of time. Therefore, it cannot allocate the hash chains to the responding agents and create personalized hash chains from beginning. To solve this issue, the naive approach will be to let the *orchestrator* agent to create the chains as soon as it finds with whom it has to contact. For example, if it initially learns that it has to contact with A_1 and A_2, it generates two or more hash chains of length n, and sends the root of these personalized hash chain to user to sign. The user generates a Merkle tree on top of the hash chain terminal values and signs the Merkle root, and sends the Merkle tree and the signature to agent A_I^*. Every time the agent creates a new hash chain, the user updates its Merkle tree and its signature. This approach, however, is not efficient and requires too many interactions between the *orchestrator* agent and the user.

Below, we give a more efficient scheme that allows the ICP to be enforced for a dynamic workflow (plan) with minimal interactions with the user. The idea is that user creates m one time signature (OTS) key pairs and generate a Merkle tree on the OTS public keys. Then, it signs the root of the Merkle tree, and sends the Merkle tree to the *orchestrator* agent together with delegating the OTS private keys to the *orchestrator* agent. On receiving these keys, the *orchestrator* agent can generate personalized hash chains for any agent A_i and it can plug in the hash chain to the Merkle tree by signing the terminal values of the hash chain using the delegated private keys. See below for more details.

One time signature (OTS) is a digital signature scheme that allows using a private and public key pair to sign a message only once. The examples of OTS are Lamport signature scheme [36], and WOTS+ signature scheme [6]. Both of these OTS schemes are hash-based stateful signatures and need secure state management. We consider that OTS consists of: (i) $OTS.Gen(\lambda)$ which takes the security parameter λ as input and outputs a private and public key pair. (ii) $OTS.Sign_{Sk}(m)$, which takes a message m and the OTS private key Sk and outputs a signature σ. (iii) $OTS.Verify_{Pk}(< m >, \sigma)$ which takes a message m, an OTS signature σ and a public key Pk and outputs 1 if the signature is valid, and 0 otherwise.

- The user U^* determines the maximum message budget Q_{tot} for *ICP*, and creates m OTS key pairs such that $Q_{tot} = n \times m$. It constructs a Merkle tree over the OTS public keys and signs the Merkle root along with the hash chain length:

  $$\sigma_{ICP}^{U^*} = \mathsf{HS.Sign}_{sk_{U^*}}(Mroot,\ n).$$

  It then sends the Merkle tree and signature to the master agent A_I^*, and delegates the OTS private keys to A_I^*.
- Upon receiving a task from U^*, agent A_I^* processes it and obtains the workflow, which identifies the first agent A_i, $i \in [1, t]$, to contact. It derives a personalized hash chain of length n using the seed:

  $$s_0 = \mathsf{PRF}(sid,\ aid_{A_I^*},\ aid_{A_i},\ addr)$$

  and extends it as $s_j = H(s_{j-1},\ j,\ sid,\ aid_{A_i})$ for $j = 1, \ldots, n$. It then signs the chain root using one of the delegated OTS private keys:

  $$\sigma_{OTS}^{A_I^*} = \mathsf{OTS.Sign}_{OTS.sk}(s_n)$$

  Secure state management is assumed to prevent reuse of any OTS key.

- When A_I^* initiates an A-session with A_i, it sends the following to A_i:
  - the Merkle root *Mroot* and user signature $\sigma_{ICP}^{U^*}$
  - the OTS public key used to sign s_n, together with a Merkle proof that this key is included in the tree signed by U^*
  - the OTS signature $\sigma_{OTS}^{A_I^*}$ on s_n,
  - the chain openings s_n and s_{n-1}, where $s_n = H(s_{n-1},\ n,\ sid,\ aid_{A_i})$
- Agent A_i verifies the following:
  - the user's signature: $\mathsf{HS.Verify}_{pk_{U^*}}(\langle Mroot,\ n\rangle,\ \sigma_{ICP}^{U^*}) = 1$,
  - the OTS signature: $\mathsf{OTS.Verify}_{OTS.pk}(\langle s_n\rangle,\ \sigma_{OTS}^{A_I^*}) = 1$,
  - the Merkle proof is valid,
  - the hash step: $s_n = H(s_{n-1},\ n,\ sid,\ aid_{A_i})$.

  If all checks pass, A_i is assured that U^* has authorized a hash chain of length n for agent A_i to fulfill the task. Agent A_i then initializes a counter $ctr_{ICP} = n$ and decrements it by one.

In the next rounds, the initiator agent just sends a hash token from the chain s_j, and the responding agent A_i verifies that $s_j = H(s_{j-1}, j, sid, aid_{A_i})$, and then decreases the counter ctr_{ICP}. When the counter ctr_{ICP} reaches 0 it checks that $s_0 = PRF(sid, aid_{A_I^*}, aid_{A_i}, addr)$. If the agent wants to continue interactions and use more hash chains, it can increment *addr* by 1, and repeat the last two steps mentioned above to start a new personalized hash chain.

ICP in the two-agent case. In the two-agent scenario, ICP becomes similar to RCP, and we deal with it in the same way using the same message count tokens used in RCP description. Therefore, the agent creates the hash chains of the length determined by the policy, and the user signs the hash terminal value. Then, agent sends the message count token and the signature to the responding agent.

D User-Agent Interaction for Multi-agent Setting

We assume the communication between the user and the master agent is local. In the agent registration subprotocol, the user has determined the ICP and RCP policies for all agents. For ICP which defines the maximum interaction budget for total requests, user has determined m and n' such that $Q_{tot} = n' \times m$, and has shared m and n' with the master agent. When the user allocates the task to the master agent A_I^*, the following subprotocol is run:

(1) **Giving the task to the agent.** The user U^* determines the task *tsk* and sends it to the master agent A_I^*.
(2) **Executing the task and obtaining the workflow.** The master agent A_I^* processes the task and derives the workflow, which specifies the list of agents $\{A_1, \ldots, A_t\}$ that A_I^* must contact to fulfill the task.
(3) **Creating the hash chain tokens.** Agent A_I^* generates m personalized hash chains. For each chain $i \in [0, m-1]$, the seed is derived as:

$$\rho_0^i = \mathsf{PRF}(sid,\ aid_{A_I^*},\ aid_{A_j},\ addr),$$

and the chain is extended as:

$$\rho_k^i = H(\rho_{k-1}^i,\ k,\ sid_j,\ aid_{A_j}), \qquad k = 1, \ldots, n',$$

where aid_{A_j} is the identity of the agent assigned to chain i according to the workflow, such that each agent receives at least one chain. Agent A_I^* then sends the hash roots $\{\rho_{n'}^i\}_{i\in[0,m-1],\ j\in[1,t]}$ to the user U^*.

(4) **Creating the Merkle tree and signing the root.** The user U^* constructs a Merkle tree over the hash roots $\{\rho_{n'}^i\}_{i\in[0,m-1],\ j\in[1,t]}$ received from A_I^*, obtaining the root *Mroot*. The user signs the root as:

$$\sigma_{ICP}^{U^*} = \mathsf{HS.Sign}_{sk_{U^*}}(Mroot,\ n')$$

The user then sends the Merkle tree and signature $\sigma_{ICP}^{U^*}$ to A_I^*.

(5) **Verifying the signature.** Agent A_I^* verifies $\sigma_{ICP}^{U^*}$ by checking:

$$\mathsf{HS.Verify}_{pk_{U^*}}\left(\langle\, Mroot,\ n'\,\rangle,\ \sigma_{ICP}^{U^*}\right) = 1$$

and stores the Merkle tree upon success.

For the dynamic workflow, note that the Merkle tree is constructed over the OTS keys and the protocol have been described in Section C.

E UC Model and Ideal Functionalities

Below we give an overview of the UC approach and the description of each ideal functionality that is needed in our analysis.

E.1 UC Approach

In the UC approach, the desired behavior of a system is specified by an *ideal functionality* $\mathcal{F}$, which describes the working of the system in ideal world. Security of a protocol π is defined by comparing the real-world execution of π in the presence of an adversary $\mathcal{A}$ with the ideal-world execution of $\mathcal{F}$, where the adversary's behavior is emulated by a *simulator* Sim. Protocol participants, $P_1, P_2 \cdots P_\ell$, are modeled by polynomially-bounded Interactive Turing Machines (ITM). A special ITM $\mathcal{Z}$, called the *environment*, models the influence of the surrounding protocol execution on π. The environment provides inputs to the participants, receives the outputs of honest parties, has access to the corrupted parties through $\mathcal{A}$, and may interact arbitrarily with $\mathcal{A}$. *The protocol π is UC-secure* if no environment $\mathcal{Z}$ can distinguish between the real-world and ideal-world execution of the protocol.

E.2 Global Clock Ideal Functionality $\mathcal{G}_{\mathsf{clk}}$

We follow the clock functionality of [12] that provides a universal reference (physical) clock. When it is queried it provides an abstract notion of the time represented by τ. The clock is monotonic and only the environment can increment it; also the simulator cannot forge this reference time.

Global clock ideal functionality $\mathcal{G}_{\text{clk}}$

maintains an integer τ corresponding to the reference time. When created, it initializes to $\tau = 0$.

- Upon receiving message $(ClockUpdate, sid)$ from $\mathcal{Z}$, update $\tau = \tau + 1$ and send $(ClockUpdate, sid, ok)$ to $\mathcal{Z}$. Ignore any clock update message sent by an other party.
- Upon receiving $(ClockRead, sid)$ from any entity return $(clock\text{-}read,\ sid,\ \tau)$ to the requester.

Figure 11: Global clock ideal functionality $\mathcal{G}_{\text{clk}}$ [12]

E.3 Secure Communication Session Ideal Functionality $\mathcal{F}_{\text{SCS}}$

This functionality captures the security requirements of the TLS channel and allows a secure communication between entities in a single protocol instance. TLS consists of two phases: handshake and message transmission. The handshake protocol aims at securely sharing uniformly distributed session keys, and the message transmission provides authenticated encryption of session messages. The following ideal functionality $\mathcal{F}_{\text{SCS}}$ [23] consists of session establishment and message sending and captures the security requirements of the TLS session which are: (i) *Uni- and bi-directional authentication (non-transferrable authentication).* In uni-directional authentication mode, this functionality allows the adversary to impersonate the initiator. However, the adversary cannot impersonate the initiator after the two parties start communicating with each other. (ii) *Confidentiality of messages.* This property is captured since adversary can only see the message of corrupted parties, and only receives a leakage $l(m)$ for messages of honest parties. The leakage function in TLS includes the message length and the TLS error messages. (iii) *Integrity.* Adversary cannot change the honest parties' messages (in practice these messages have a sequence number that comes from the application layer to prevent changing their order). Adversary can only send corrupted messages to corrupted parties of its choice. We consider that $\mathcal{F}_{\text{SCS}}$ captures the security properties of the post quantum TLS channel (that uses post-quantum secure primitives) as well.

E.4 Global Random Oracle Ideal Functionality $\mathcal{G}_h$

The random oracle functionality $\mathcal{G}_h$ [9] has a $query$ interface, and responds to all queries with uniformly random sampled values $r \leftarrow \{0,\ 1\}^{l(\kappa)}$, and outputs the same value for the same query. $\mathcal{G}_h$ allows the adversary to program the random oracle to its desired values. At the same time, to prevent the adversary from programming collisions, it allows the parties to check whether a query has been programmed or not and drop that value if so. Additionally, the adversary can observe all the adversarial queries made from a wrong session L_{sid}.

E.5 Certification Authority Ideal Functionality $\mathcal{F}_{\text{CA}}$

We follow [11] for the ideal functionality of $\mathcal{F}_{\text{CA}}$ (see Figure 14). This ideal functionality is bound to a single identity via sid. $\mathcal{F}_{\text{CA}}$ accepts only the first registered message and does not allow for modification or revocation. $\mathcal{F}_{\text{CA}}$ does not perform any computation on the received message and only acts as the public bulletin board.

Functionality F_{scs}, proceeds as follows, when parameterized by a leakage function $l : \{0,1\}^* \rightarrow \{0,1\}^*$.

- Upon receiving an input $(establish\text{-}session, sid, IDI)$ from some party, where $IDI \in (\perp, I)$, record IDI as initiator, and send the message to the adversary. Upon receiving input $(establishsession, sid, R)$ from some party, record R as responder, and forward the message to the adversary.
- Upon receiving a value $(impersonate, sid)$ from the adversary, do: If $(IDI = \perp)$, check that no ready entry exists, and record the adversary as initiator. Else ignore the message.
- Upon receiving a value $(send, sid, m, P')$ from party P, which is either initiator or responder, check that a record (sid, P, P') exists, record ready (if there is no such entry) and send $(sent, sid, l(m))$ to the adversary and a private delayed value $(receive, sid, m, P)$ to P'. Else ignore the message. If the sender is corrupted, then disclose m to the adversary. Next, if the adversary provides $\hat{m}$ and no output has been written to the receiver, then send $(send, sid, \hat{m}, \hat{P})$ to the receiver unless $\hat{P}$ is an identity of an uncorrupted party.

Figure 12: The secure communication session ideal functionality, $\mathcal{F}_{\text{SCS}}$

Functionality $\mathcal{G}_h$

Shared functionality $\mathcal{G}_h$ is globally available to all participants. It takes as input queries $q \in \{0,1\}^*$ and outputs values $r \in \{0,1\}^k$. Internally, it stores initially empty sets L, L_p, and a set L_{sid} for all sessions sid.

Query

– Upon receiving $(query,\ sid,\ q)$ from a party of session sid' proceed as follows:

- If $(sid,\ q,\ r) \in L$ respond with $(query,\ q,\ r)$.
- If $(sid,\ q,\ r) \notin L$, samples $r \in \{0,1\}^k$, store $(sid,\ q,\ r)$ in Q and respond with $(query,\ q,\ r)$.
- If the query is made from a wrong session $(sid \neq sid')$, store $(q,\ r)$ in L_{sid}.

Program

– Upon receiving message $(program,\ sid,\ q,\ r)$ by the adversary, check if $(sid,\ q,\ r')$ is defined in L. If this is the case, abort. Otherwise, if $r \in \{0,1\}^k$ store $(sid,\ q,\ r)$ in Q and $(sid,\ q)$ in L_p.

– Upon receiving $(isPrgrmd,\ q)$ from a party of session sid, check if $(sid,\ q)$ in L_p. If this is the case respond with $(isPrgmd,\ 1)$.

Observe

– Upon receiving message $(observe)$ from the adversary $\mathcal{A}$ of session id sid respond with $(observe,\ L_{sid})$.

Figure 13: Global restricted programmable and observable random oracle functionality $\mathcal{G}_h$ [9], provides three interfaces *Query*, *Program*, and *Observe* to participants.

E.6 Secure Registration Ideal Functionality F_{Reg}

We define F_{Reg} to capture the secure registration of the users and the agents (see Figure 15). The user U can choose a specific provider TA

Functionality F_{CA}

- Upon receiving the first message $(Register, sid, v)$ from party P, send $(Registered, sid, v)$ to the adversary; upon receiving ok from the adversary, and if $sid = P$, and this is the first request from P, then record the pair (P, v).
- Upon receiving a message $(Retrieve, sid)$ from party P', send $(Retrieve, sid, P')$ to the adversary, and wait for an ok from the adversary. Then, if there is a recorded pair (sid, v) output $(Retrieve, sid, v)$ to P'. Else, output $(Retrieve, sid, \perp)$ to P'.

Figure 14: The certification authority ideal functionality, $\mathcal{F}_{\text{CA}}$

Secure registration ideal functionality F_{Reg}, has access to the list of registered providers L_{TA}, and holds a list L_{UReg} and L_{AReg} containing the identity of the registered user and the registered agents respectively.

- Upon receiving an input $(RegisterUser, sid, U, Pwd, TA)$ from some party U, check $TA \in L_{TA}$, also check U has not been registered before, send $(RegisterUser, sid, TA, U)$ to the adversary; upon receiving ok from the adversary store (TA, U, Pwd) in L_{UReg}.
- Upon receiving an input $(RegisterAgent, sid, A, attr, Pwd, TA)$ from party U, where $attr$ is attribute of A, check that (TA, U, Pwd) exists and hence U is a registered user with TA, also check that $(U, A, attr)$ does not exist in the record, then send $(RegisterAgent, sid, U, A)$ to the adversary; upon receiving ok from the adversary, store $(U, A, attr, -, -, -)$ in L_{AReg}.
- Upon receiving an input $(AgentPolicy, sid, A, attr, CP_A, Pwd, TA)$ from U, where $CP_A = \{[Send/Receive, A_i, Budget_i]\}$ specifies the identity of agents who can contact with A, and the number of times each of those agents A_i can contact A, i.e. $Budget_i$, it checks whether the user is registered and $(TA, U, Pwd) \in L_{UReg}$, checks whether $(U, A, attr, \cdot, \cdot) \in L_{AReg}$ exists and hence A is already registered. Then send $(AgentPolicy, sid, U)$ to the adversary, and update $(U, A, attr, CP_A)$ in L_{AReg}.

Figure 15: Secure user and agent registration ideal functionality $\mathcal{F}_{\text{Reg}}$

to register with and choose a password Pwd that will be stored with its identity (capturing that the user holds a certified public key) and will be used for authentication when registering the agents. It also allows a user who has registered with an authorized TA to register its agent A with attribute $attr$. The agent information consisting of the user identity, the agent identity, and the agent attribute are stored in the list of registered agents L_{AReg}. The adversary learns about the identity of the registered agents (we assume they are publicly known). F_{Reg} also allows the user U to add or update a contact policy for its agent. The contact policy $CP_A = \{[\text{Send/Receive}, A_i, \text{Budget}_i]\}$ specifies which agents A_i may send requests to, or receive requests from, A, together with the corresponding contact budget Budget_i. The adversary is only informed about the call but nothing beyond that, which captures the fact that the communication is over PQ-TLS channel and the adversary can only learn if the user U has contacted the provider without knowing for which of the agents it has contacted nor for what purpose, that is, adding a policy or updating a policy.

E.7 A-session Authorization Ideal Functionality $\mathcal{F}_{\text{A-auth}}$

It is run between an agent A and the Provider TA, and ensures that an agent is authorized (according to the user-defined policies CP_A) to establish an A-session with another agent (see Figure 16).

A-session Authorization ideal functionality $\mathcal{F}_{\text{A-auth}}$, holds a list L_{AReg} containing the identity and attribute of the registered agents, and their contact policy CP_A of all registered agents, and a counter CTR set that keeps the counter states for enforcing the budget.

- Upon receiving an input $(GetAuthorization, sid, A, attr, A', attr', TA)$ from A, the ideal functionality checks whether $(A, attr) \in L_{AReg}$ and $(A', attr') \in L_{AReg}$. It then retrieves CP_A and $CP_{A'}$ and checks whether $(A, A') \in CP_A \cap CP_{A'}$. If yes, it finds the counter $ctr_{A,A'} \in CTR$ which has ben initialized with $B = min\{Budget_A, Budget_{A'}\}$ the first time, and checks whether $ctr_{A,A'} \leq B$. If so, it decrements $ctr_{A,A'}$ by one, and sends $(Allowed, sid, A, attr, A', attr')$ to the agent A, the TA, and the adversary, and stores $(Allowed, sid, A, attr, A', attr')$ in L_{Auth}.

Figure 16: A-session Authorization ideal functionality $F_{A\text{-}auth}$

E.8 Secure A-session Ideal Functionality $\mathcal{F}_{\text{Ases}}$

The secure A-session ideal functionality, shown in Figure 17, involves a user and two agents, A_I and A_R. It captures the security requirements of an A-session, including *task-msg* confidentiality, participant authentication, *task-msg* integrity, accountability, and policy enforcement with respect to *task-msg* and time budgets. In this functionality, the users specify their agents' policies, and the user of A_I provides the task to be performed. A_I acts as the initiator and contacts A_R multiple times to complete the given task.

The task, in our model, determines when the session starts and when it terminates. We model task execution as a stateful request-response computation between the agents. The initiator-side algorithm ExeRes takes the task tsk, the previous response, and the initiator's code as input, and outputs the next request. The responder-side algorithm ExeReq takes a request and the responder's code as input, and outputs a response. The responder need not know the underlying task tsk.

The algorithm ExeReq may itself be stateful and interactive: the responding agent may interact with other agents before producing its response to A_I.

The ideal functionality is parameterized by a leakage function $\ell()$ on the request/response transcript and the *task-msg* and time budgets. If one agent is corrupted, $\ell()$ leaks the budgets fully, reflecting the policy information exchanged during an A-session. If both agents are honest, $\ell()$ leaks only unavoidable transcript metadata, such as message lengths and execution times.

The ideal functionality handles requests and responses internally, since the number of rounds may depend on the task. When the initiator A_I is honest, the adversary does not know the task; hence, without the ideal functionality running the task internally and suitable leakage the simulator may be unable to reproduce the observable round structure of the real execution, allowing the environment to distinguish the two worlds. Following the internal simulation of rounds and leakage-based approach of [4, 21], the functionality simulates the task execution internally and therefore leaks specified metadata in each round, enabling the simulator to match the observable transcript while preserving the intended communication-security guarantees.

$\mathcal{F}_{\text{Ases}}$ captures the stated security requirements as follows:

- *Confidentiality of* task-msg. If both agents are honest, the adversary learns only the leakage about the requests and responses, as defined by the leakage function $\ell()$.

- *Authentication.* Only registered and authorized agents can participate in an A-session; the adversary cannot impersonate an honest entity.
- *Integrity.* Requests and responses are simulated internally by $\mathcal{F}_{\text{Ases}}$ and cannot be modified, injected, replayed, or reordered by the adversary.
- *Accountability.* If a corrupted party sends an invalid, malformed, or policy-violating *task-msg*, $\mathcal{F}_{\text{Ases}}$ can identify the corrupted party whose behavior causes an abort through the adversary, and attribute that to the corrupted party.
- *Policy enforcement and authorization.* $\mathcal{F}_{\text{Ases}}$ receives the two agents' A-session policies, including their *task-msg* and time budgets, and enforces the effective session policy obtained from their intersection. It maintains the relevant counters and timers, and terminates the session once a budget is exhausted, the time bound expires, or a corrupted party triggers an abort (capturing the early termination of session by a corrupted party).

The *authorization soundness* in Section 5 is guaranteed by the composition of $\mathcal{F}_{\text{A-auth}}$ and $\mathcal{F}_{\text{Ases}}$.

Note that our protocol also ensures *Forward security* for an A-session which means that compromise of long-term credentials does not compromise previously established A-session keys. While our protocol provides this property, we leave modeling and proving this property for future work.

E.9 The Secure Multi-Agent Composite Session (C-session) Ideal functionality, $\mathcal{F}_{\text{Cses}}$ with Static Workflow

This ideal functionality allows a user to give a task to an *orchestrator* agent A_I^*, where A_I^* contacts a set of responding agents A_i (according to a static work plan) to fulfill the task (see Figure 18). This ideal functionality ensures the security properties listed in $\mathcal{F}_{\text{Ases}}$ in a similar way but in the communication across mutiple responding agents (see description of $\mathcal{F}_{\text{Ases}}$). These properties are: *task-msg* confidentiality, participant authentication, *task-msg* integrity, accountability, and policy enforcement with respect to *task-msg* and time budgets of A_I^* and the set of A_i. In $\mathcal{F}_{\text{Cses}}$ the users determine the policy for their agents at the beginning of the session. The agent policy is in the form of $[(A, A'), Q_{A,A'}, \Delta_{A,A'}]$ and consists of the interaction budget $Q_{A,A'}$ and time budget $\Delta_{A,A'}$. $[(A, \perp), Q_{tot}, \Delta_{tot}]$ represents the policy of *orchestrator* agent (i.e., ICP).

We highlight three points in our model: (i) Similar to the A-session functionality, we model the task execution as a stateful interactive computation, in which A_I^* acts as initiator and contacts with other agents A_i several times in order to complete the given task. We only consider one-hop interactions, which means A_I^* runs $ExeRes()$, and contact another agent A_i back and forth, but we do not model the case where A_i interacts with other agents to respond the requests from A_I^* as it makes it very complex for the analysis. Without loss of generality, we assume that the A_i uses the reactive function $ExeReq()$ to respond to a request. This function/ computation by itself is an stateful interactive function which allows A_i to interact with other agents and respond to A_I^* at the end. (ii) We consider a static workflow for task execution, where the *orchestrator* agent A_I^* first determines the *schedule* consisting of the list of agents $(A_i, attr_i)$ that should be contacted based on the given task. (iii) The ideal functionality simulates the handling of requests and responses internally since the number of requests and responses depend on the given task, and the adversary who has not corrupted the *orchestrator* agent A_I^* and does not know the task cannot simulate the number of rounds of requests and response and the agents who should be contacted with correctly. This will enable the environment to distinguish the ideal world from the real world. Therefore, we follow the approach of [4, 21] and consider that the ideal functionality simulate the request and response process internally, enforces the policy, and leak some information to the adversary. With this consideration, we only need to show that the simulator can simulate each round correctly even if only a negligible information is leaked to it.

The Secure A-session ideal functionality $\mathcal{F}_{\text{Ases}}$, is defined for a given task tsk, and interacts with a global clock functionality $\mathcal{G}_{\text{clk}}$, and it is parametrized by a leakage function $\ell(\cdot) : \{0,1\}^* \times \{0,1\}^{|Q|} \times \{0,1\}^{|\Delta|} \rightarrow \{0,1\}^*$ (note that sid includes $\mathcal{G}_h(tsk)$). $\mathcal{F}_{\text{Ases}}$ holds the list of registered agents L_{AReg} and the list of agents that are authorized to establish an A-session L_{Auth}.

- Upon receiving $(AgentPolicy, sid, APolicy)$ from the user U, where $Apolicy$ consists of the tuple of the form $[(A, A'), Q_{A,A'}, \Delta_{A,A'}]$, store $(sid, APolicy)$ in L_{APo}. If A is corrupted and later it receives $(ChangePolicy, sid, APolicy')$ from the adversary replace $APolicy$ with $Apolicy'$ in L_{APo}.
- Upon receiving a $(Task, sid, tsk)$ from $(A_I, attr_I)$, where tsk is given to agent A_I with attribute $attr_I$ (if $tsk = \perp$ abort), it checks $(A_I, attr_I) \in L_{AReg}$, and stores $(sid, tsk, A_I, attr_I)$. Then, run $(A_R, attr_R, req_1) \leftarrow ExeRes(A_I, tsk, \perp)$, where $Execute$ runs the code of A_I on the given task tsk and outputs (i) the identity and attribute $(A_R, attr_R)$ of the agent that A_I has to connect to, (ii) the first request to be sent from A_I denoted by req_1. Then it checks $(A_R, attr_R) \in L_{AReg}$, if not aborts. Next, retrieve and parse $APolicy$ from L_{APo} for both agents and retrieve $[(A_I, A_R), Q_I, \Delta_I]$ and $[(A_R, A_I), Q_R, \Delta_R]$. Then, set $Q = min\{Q_I, Q_R\}$ and $\Delta = min\{\Delta_I, \Delta_R\}$. Next compute $T_{exp} = \tau + \Delta$, and start a counter ctr for interaction limit. Then, simulates the task execution internally as below by starting with *Request handling* where $req_0 = \perp$:
  - **–Request handling:** upon receiving $(Request, sid, A_I, attr_I, req_i)$ from A_I with attribute $attr_I$, check $(AuthorizationAllowed, \cdot, A_I, attr_I, A_R, attr_R) \in L_{Auth}$; if so then send $(ClockRead, sid)$ to $\mathcal{G}_{\text{clk}}$ and get the time τ. Then check whether $ctr \leq Q$, and increment ctr by one; Then, check $\tau \leq T_{exp}$. If the checks do not pass, abort and inform the adversary. Otherwise, run $res_i \leftarrow ExeReq(A_I, req_i)$, and send $(Response, sid, A_R, attr_R, \ell(res_i, Q_R, \Delta_R))$ to the adversary (if A_I or A_R is corrupted $\ell()$ leaks Q_R and Δ_R to the adversary). Upon receiving abort from the adversary, abort and output the identity of the corrupted agent, else proceed to *response handling*.
  - **–Response handling:** upon receiving $(Response, sid, A_R, attr_R, res_i)$ from A_R with attribute $attr_R$, check whether $ctr \leq Q$, and increment ctr by one; Then, check $\tau \leq T_{exp}$. If the checks do not pass, abort and inform the adversary. Otherwise, run $(req_i, result) \leftarrow ExeRes(A_R, tsk, res_i)$. If $req_i = \perp$, then output $result$ and terminate. Else send $(Request, sid, A_I, attr_I, \ell(req_i, Q_I, \Delta_I))$ to adversary (if A_I or A_R is corrupted $\ell()$ leaks Q_I and Δ_I to the adversary). Upon receiving $terminate$ from adversary, output res_i and terminate. Upon receiving abort from the adversary, abort and output the identity of the corrupted agent, else proceed to *request handling*.

Figure 17: The secure A-session ideal functionality, $\mathcal{F}_{\text{Ases}}$

E.10 The Secure Multi-agent Composite Session (C-session) Ideal Functionality, $\mathcal{F}_{\text{Cses}}$ with Dynamic Workflow

This ideal functionality allows the users to determine the policy for their agents, where the agent policy is in the form of $[(A, A'), Q_{A,A'}, \Delta_{A,A'}]$ and consists of the interaction budget $Q_{A,A'}$ and time

The secure multi-agent composite session (C-session) ideal functionality, $\mathcal{F}_{\text{Cses}}$, is defined for a task tsk and interacts with a global clock functionality $\mathcal{G}_{\text{clk}}$, and it is parametrized by a leakage function $\ell(\cdot) : \{0,1\}^* \times \{0,1\}^{|\mathcal{Q}|} \times \{0,1\}^{|\Delta|} \rightarrow \{0,1\}^*$. This ideal functionality has access to the list of registered agents L_{AReg}, and the list of agents that are authorized to establish an A-session L_{Auth}.

- Upon receiving $(AgentPolicy, sid, APolicy)$ from the user U, where $Apolicy$ consists of the tuple of the form $[(A, A'), Q_{A,A'}, \Delta_{A,A'}]$, store $(sid, APolicy)$ in L_{APo}. If A is corrupted and later it receives $(ChangePolicy, sid, APolicy')$ from the adversary replace $APolicy$ with $Apolicy'$ in L_{APo}.
- Upon receiving $(Task, sid, tsk, A_I^*, attr^*)$ from user U^*, where tsk is the given task from user U^* to *orchestrator* agent A_I^* with attribute $attr^*$, then store $(sid, tsk, U^*, A_I^*, attr^*)$ (if $tsk = \perp$ abort). Next, run $(schedule, req_i^1) \leftarrow ExeRes(A_I^*, tsk, \perp)$, where $ExeRes$ runs the code of A_I^* on the given task tsk and outputs the first request req_i^1 and the schedule which consists of the identity $[(A_1, attr_1), \cdots, (A_t, attr_t)]$ of all the agents that should be contacted with (and the order of invocation). Then, check whether $(A_i, attr_i) \in L_{AReg}\ \forall i$, and retrieve and parse $APolicy$ from L_{APo} for A_I^*, that is, $[(A_I^*, \perp), Q_{tot}, \Delta_{tot}]$. Next compute $T_{exp} = \tau + \Delta_{tot}$. Also, start a counter ctr for interaction limit. It then simulates the task execution internally as below:
  - **Request handling:** upon receiving $(Request, sid, A_i, req_i^k)$ from A_I^* with attribute $attr^*$, check $(AuthorizationAllowed, \cdot, A_I^*, attr^*, A_i, attr_i) \in L_{Auth}$; if so then send $(ClockRead, sid)$ to $\mathcal{G}_{\text{clk}}$ and get the time τ. If this is the first time a request is received from A_I^*, retrieve and parse $APolicy$ from L_{APo} for agent A_i and retrieve $[(A_I^*, A_i), Q_{*,i}, \Delta_{*,i}]$. Then, check $\Delta_{*,i} < \Delta_{tot}$ and compute $T'_{exp} = \tau + \Delta_{*,i}$. Check that $T'_{exp} < T_{exp}$ and store T'_{exp}. Else, abort. Also, start a counter ctr_i for interaction limit and increment both ctr and ctr_i. Else, if this is not the first message, check whether $ctr_i \leq Q_{*,i}$ and $\tau < T'_{exp}$, and then increment ctr_i by one. If the checks do not pass, abort and inform the adversary. Otherwise, run $res_i^k \leftarrow ExeReq(A_i, req_i^k)$, and send $(response, sid, A_i, \ell(res_i^k, Q_{*,i}, \Delta_{*,i})$ to the adversary. Upon receiving abort from the adversary, abort and output the identity of the corrupted agent, else proceed to *response handling*.
  - **Response handling:** upon receiving $(Response, sid, A_I^*, res_i^k)$ from A_i with attribute $attr_i$, check whether $ctr \leq Q_{tot}$, and increment ctr by one; Then, check $\tau \leq T_{exp}$. If the checks do not pass, abort and inform the adversary. Otherwise, run $(A_j, attr_j, req_j^{k+1}, result) \leftarrow ExeRes(A_I^*, tsk, res_i^k)$. If $A_j = \perp$, then output $result$ and terminate. Else, send $(Request, sid, A_j, \ell(req_j^{k+1}, Q_{tot}, \Delta_{tot}))$ to adversary. Upon receiving abort from the adversary abort and output the identity of the corrupted agent, else proceed to *response handling*.

Figure 18: The secure multi-agent composite session (C-session) ideal functionality, $\mathcal{F}_{\text{Cses}}$ with static workflow

budget $\Delta_{A,A'}$. The policy $[(A, \perp), Q_{tot}, \Delta_{tot}]$ represents the policy of a *orchestrator* agent (i.e., ICP). $\mathcal{F}_{\text{Cses}}$ ensures: *task-msg* confidentiality, participant authentication, *task-msg* integrity, accountability, and policy enforcement with respect to *task-msg* and time budgets of A_I^* and the set of A_i, across multiple agents. It ensures these security properties in a similar way described in $\mathcal{F}_{\text{Ases}}$ but in the communication across many responding agents (see description of $\mathcal{F}_{\text{Ases}}$).

In $\mathcal{F}_{\text{Cses}}$, a user gives a task tsk to an *orchestrator* agent A_I^* with attribute $attr^*$ which orchestrates the communication with other agents (see Figure 19). The *orchestrator* agent can be corrupted in which case there is no guarantee that it performs the task correctly (we allow the adversary to replace the task which affects the order of agent calls and the requests it makes, and also to replace the result with adversarial ones). If the *orchestrator* agent A_I^* is honest, the ideal functionality execute the task in a trusted way, which means that it first gets identity of the first agent that needs to be called and the first request by running the reactive function $ExeRes(A_I^*, tsk, \perp)$ on the given task tsk using the code of the agent A_I^*. It, then, internally simulates the request sending and receiving of the agents, and leaks some information about each request and response to the adversary. We also let the adversary to terminate the request and response handling to capture the fact that in case the interacting agents are corrupted they may abort earlier and then no results will be returned to the user.

Note that we need to clarify three points: (i) We model the task execution as an stateful interactive computation, in which A_I^* acts as initiator and contacts with other agents A_i several times in order to complete the given task. We only consider one-hop interactions, which means A_I^* runs $ExeRes()$, and contact another agent A_i back and forth, but we do not model the case where A_i interacts with other agents to respond the requests from A_I^* as it makes it very complex for the analysis. Without loss of generality, we assume that the A_i uses the reactive function $ExeReq()$ to respond to a request. This function/ computation by itself in an stateful interactive function which allows A_i to interact with other agents and respond to A_I^* at the end. (ii) We consider a dynamic workflow for task execution, where the *orchestrator* agent A_I^* first determines the first contacting agent $(A_i, attr_i)$ based on the given task and then based on the responses that it receives from this agent it decides the next agent that should be contacted. Therefore, we consider that $ExeRes()$ always output an identity and the next request. (iii) The ideal functionality simulates the handling of requests and responses internally since the number of requests and responses depend on the given task, and the adversary who has not corrupted the *orchestrator* agent A_I^* and does not know the task cannot simulate the number of rounds of requests and response and the agents who should be contacted with correctly. This will enable the environment to distinguish the ideal world from the real world. Therefore, we follow the approach of [4, 21] and consider that the ideal functionality simulate the request and response process internally, enforces the policy, and leak some information to the adversary. With this consideration, we only need to show that the simulator can simulate each round correctly even if only a negligible information is leaked to it.

F Security Analysis

In this section, we give the security analysis of MAGIQ in multiple steps as described in Figure 6.

F.1 Main Theorem

The following theorem gives the security of MAGIQ:

Theorem F.1. *MAGIQ (Section 4) UC-realizes* $\mathcal{F}_{\text{Reg}}$, $\mathcal{F}_{A\text{-auth}}$, $\mathcal{F}_{\text{Ases}}$, *and* $\mathcal{F}_{\text{Cses}}$ *in the* $(\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h)$*-hybrid model. The cryptographic building blocks of MAGIQ are (i) a digital signature scheme that is EUF-CMA, and (ii) hash function, HMAC and PRF that are modeled by global random oracles.*

(It is sufficient for the signature scheme that are used by all MAGIQ participants to be EUF-CMA in standard model).

Proof.

Proof sketch. We construct the protocol $\pi_{MAGIQ}^{\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h}$ in the $(\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h)$-hybrid model and prove Theorem 5.1 through a sequence of lemmas, each capturing the security of one MAGIQ

The secure multi-agent composite session (C-session) ideal functionality, $\mathcal{F}_{\text{Cses}}$, is defined for a task tsk and interacts with a global clock functionality $\mathcal{G}_{\text{clk}}$, and it is parametrized by a leakage function $\ell(\cdot) : \{0,1\}^* \times \{0,1\}^{|Q|} \times \{0,1\}^{|\Delta|} \rightarrow \{0,1\}^*$. This ideal functionality has access to the list of registered agents L_{AReg}, and the list of agents that are authorized to establish an A-session L_{Auth}.

- Upon receiving $(AgentPolicy, sid, APolicy)$ from the user U, where $Apolicy$ consists of the tuple of the form $[(A, A'), Q_{A,A'}, \Delta_{A,A'}]$, store $(sid, APolicy)$ in L_{APo}. If A is corrupted and later it receives $(ChangePolicy, sid, APolicy')$ from the adversary replace $APolicy$ with $Apolicy'$ in L_{APo}.
- Upon receiving $(Task, sid, tsk, A_I^*, attr^*)$ from user U^*, where tsk is the given task from user U^* to *orchestrator* agent A_I^* with attribute $attr^*$, then store $(sid, tsk, U^*, A_I^*, attr^*)$ (if $tsk = \perp$ aborts). Next, run $(A_i, attr_i, req_i^1) \leftarrow ExeRes(A_I^*, tsk, \perp)$ where $ExeRes$ runs the code of A_I^* on the given task tsk and outputs the first request req_i^1 and the identity $(A_i, attr_i)$ of the first agent that needs to be contacted. Then, retrieve and parse $APolicy$ from L_{APo} for A_I^*, that is, $[(A_I^*, \perp), Q_{tot}, \Delta_{tot}]$. Next compute $T_{exp} = \tau + \Delta_{tot}$. Also, start a counter ctr for interaction limit. It then simulates the task execution internally as below:
  - **Request handling:** upon receiving $(Request, sid, A_i, req_i^k)$ from A_I^* with attribute att^*, check $(AuthorizationAllowed, \cdot, A_I^*, attr^*, A_i, attr_i) \in L_{Auth}$; if so then send $(ClockRead, sid)$ to $\mathcal{G}_{\text{clk}}$ and get the time τ. If this is the first time a request is received from A_I^*, retrieve and parse $APolicy$ from L_{APo} for agent A_i and retrieve $[(A_I^*, A_i), Q_{*,i}, \Delta_{*,i}]$. Then, check $\Delta_{*,i} < \Delta_{tot}$ and compute $T'_{exp} = \tau + \Delta_{*,i}$. Check that $T'_{exp} < T_{exp}$ and store T'_{exp}. Else, abort. Also, start a counter ctr_i for interaction limit and increment both ctr and ctr_i. Else, if this is not the first message, check whether $ctr_i \leq Q_{*,i}$ and $\tau < T'_{exp}$, and then increment ctr_i by one. If the checks do not pass, abort and inform the adversary. Otherwise, run $res_i^k \leftarrow ExeReq(A_i, tsk, req_i^k)$, and send $(Response, sid, A_i, \ell(res_i^k, Q_{*,i}, \Delta_{*,i}))$ to the adversary. Upon receiving abort from the adversary abort and output the identity of the corrupted agent, else proceed to *response handling*.
  - **Response handling:** upon receiving $(Response, sid, A_I^*, res_i^k)$ from A_i with attribute $attr_i$, check whether $ctr \leq Q_{tot}$, and increment ctr by one; Then, check $\tau \leq T_{exp}$. If the checks do not pass, abort and inform the adversary. Otherwise, run $(A_j, attr_j, req_j^{k+1}, result) \leftarrow ExeRes(A_I^*, tsk, res_i^k)$. If $A_j = \perp$, then output $result$ and terminate. Else, send $(Request, sid, A_j, \ell(req_j^{k+1}, Q_{tot}, \Delta_{tot}))$ to adversary. Upon receiving abort from the adversary abort and output the identity of the corrupted agent, else proceed to *response handling*.

Figure 19: The secure multi-agent composite session (C-session) ideal functionality, $\mathcal{F}_{\text{Cses}}$ with dynamic workflow

protocol. These protocols are executed sequentially: an agent may participate in an A-session or a C-session only after registration, and an initiating agent may start the session only after obtaining authorization from the Provider. The lemmas are stated below.

Hybrid world. We consider a hybrid protocol $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h}$. This means that we replace the PQ-TLS channel in the protocol with calls to $\mathcal{F}_{\text{SCS}}$ Note that the protocol (specially the A-session) may run over multiple instances of $\mathcal{F}_{\text{SCS}}$. Additionally, we assume parties have access to a certificate authority ideal functionality $\mathcal{F}_{\text{CA}}$ (that allows registering the identities by a trusted party and has been used in the security of TLS as well [23]), the global clock ideal functionality $\mathcal{G}_{\text{clk}}$ (that captures the reference clock) and the global random oracle ideal functionality $\mathcal{G}_h$ (that models the hash functions). We consider the adversary $\mathcal{A}$ to be static which means it corrupts the parties only before the protocol starts.

Ideal world. In the ideal world, we consider the ideal functionality $\mathcal{F}$ that captures the security of the protocol and interacts with a dummy user U_I, and two agents A_I and A_R, and a Provider and

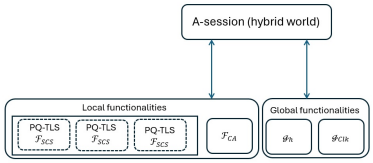


Figure 20: The A-session in the two-agent MAGIQ protocol

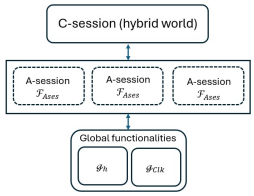


Figure 21: The C-session in the multi-agent MAGIQ protocol

uses the global clock ideal functionality $\mathcal{G}_{\text{clk}}$ and the global random oracle functionality $\mathcal{G}_{\text{clk}}$ as its subroutine (note that in our analysis, we only consider one protocol at a time, the entities that are involved in that protocol and the ideal functionality $\mathcal{F}$ that captures the security of that protocol.) Parties receive their inputs from the environment $\mathcal{Z}$ and relay them to $\mathcal{F}$. When they receive their output from $\mathcal{F}$, they just pass it to $\mathcal{Z}$. We consider a simulator Sim that is the ideal world adversary which interacts with $\mathcal{F}$, and its goal is to simulate the protocol correctly such that $\mathcal{Z}$ cannot distinguish the ideal world protocol execution from the hybrid world. Sim has access to the code of the honest parties, some times denoted by Sim^A for clarity, and can perform the computations internally. Sim can corrupt parties and in this case it gets oracle access to their codes.

F.2 Security Analysis of the Registration Protocol

In this section, we show that the user and agent registration protocol, described in Appendix **??**, UC realizes the registration ideal functionality $\mathcal{F}_{\text{Reg}}$.

Lemma F.2. *The registration protocol of $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h}$ for the user and agent (described in Appendix **??**) UC realizes the registration ideal functionality $\mathcal{F}_{\text{Reg}}$ against static adversaries.*

Proof. We analyze the security of the registration protocol considering that the user and provider are honest. Note that we have considered the provider to be semi-honest but we do not model the leakage against the provider since our goal is to only limit the information that provider can see from the agents communication, and we do not provide any cryptographic guarantee against a semi-honest provider. Below, we give the description of simulator Sim.

Simulating the provider. Sim generates the identity and PQ-TLS key pairs (sk_{TS}, pk_{TA}), and $(sk_{TA}^{tls}, pk_{TA}^{tls})$ for the provider, it also simulates $\mathcal{F}_{\text{CA}}$ by storing the identities of the provider (TA, pk_{TA}) and (TA, pk_{TA}^{tls}). Note that Sim also simulates the provider's

certificates $Cert_{TA}^{tls}$ and $Cert_{TA}^{ID}$ by signing its public keys using a generated key pair for the CA.

Simulating the user registration. Upon receiving ($RegisterUser$, sid, TA, U), Sim generates the identity and PQ-TLS key pairs (sk_U, pk_U), and (sk_U^{tls}, pk_U^{tls}) for the user, it also simulates $\mathcal{F}_{\text{CA}}$ by storing the identities of the user (U, pk_U) and (U, pk_U^{tls}). Then, it creates certificates for user's public keys $Cert_U^{tls}$, and $Cert_U^{ID}$ by signing the user's public keys using a CA private key (generated by Sim). Sim also chooses a random value as password Pwd' and stores it, sends ok to $\mathcal{F}_{\text{Reg}}$, and sends a confirmation to the user.

Simulating the agent registration.

- Upon receiving ($RegisterAgent, sid, U, A$) from $\mathcal{F}_{\text{Reg}}$, Sim generates PQ-TLS key pairs (sk_A, pk_A), and (sk_A^{tls}, pk_A^{tls}) for the agent, and simulates $\mathcal{F}_{\text{CA}}$ by storing the identity of the agent (A, Pk_A^{tls}). Then, it signs the identity public key of the agent using the user's simulated keys, that is, σ_{ID}^{U}. It also simulates the agent's metadata, and signs the agent's information σ_A^U. using the user's key. Then, it sends ok to $\mathcal{F}_{\text{Reg}}$, and sends the user the confirmation signature σ_A^{TA} which is generated on the agent's info.
- Upon receiving ($AgentPolicy, sid, U$) from $\mathcal{F}_{\text{Reg}}$, Sim generates a contact policy CP_A which is initially empty and stores it.

Sim fully simulates the user and agent protocol, since in this case, the user and provider are honest and $\mathcal{Z}$ only sees the output of the honest parties. The output of user in this protocol is a confirmation message and a signature σ_A^{TA} that is simulated by Sim and it is indistinguishable from the real world values. We give the hybrid games for the in-distinguishability of the hybrid world from the ideal world in the full version of the paper.

F.3 Security Analysis of Agent Discovery

In this section, we show that the agent discovery protocol described in Section 4 UC realizes the A-session authorization ideal functionality $\mathcal{F}_{\text{A-auth}}$.

Lemma F.3. *The agent discovery protocol of $\pi_{MAGIQ}^{\mathcal{F}_{\text{SCS}},\mathcal{F}_{\text{CA}},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ (described in Section 4.4) UC realizes the A-session authorization ideal functionality $\mathcal{F}_{\text{A-auth}}$ against static adversaries.*

Proof. The agent discovery protocol is run between an agent and a provider. The provider is honest but the agent can be corrupted. We therefore consider two case: (i) the provider and agent are honest, and (ii) the provider is honest but the agent is corrupted. In the following, we give the description of the simulator Sim in this protocol considering that the registration protocols have been done as described in the previous section:

Simulating PQ-TLS connection. If both entities are honest, this step is not needed since environment does not have access to it. If the agent is corrupted, Sim simulates $\mathcal{F}_{\text{SCS}}$ itself, and respond to the agent similar to $\mathcal{F}_{\text{SCS}}$.

Simulating the agent's authorization for A-session. We consider the following cases:

- *Case (i) Agent A is honest.* Upon receiving ($Allowed, sid, A, attr, A', attr'$), Sim simulates the information of agent A' consisting of metadata, the endpoint descriptor, certificates, and user U' signatures on the identity and information of A' using random values, and generating private and public key pairs for A'. Then, it retrieves the agent A' identity and PQ-TLS public keys (note Sim can retrieve correct values since it simulates $\mathcal{F}_{\text{CA}}$ and we assume the registration protocol have been run beforehand) and generates σ_{ac}^{TA} on the A' information and public keys of A. It then returns the A' information and σ_{ac}^{TA} to agent A.
- *Case (ii) Agent A is corrupted.* In this case, Sim waits to receive ($aid_A, aid_{A'}$) from A and uses aid_A (which incorporates Aid_U) and aid_U received from A, rather than randomly generated values for user and agent identities. It retrieves the information of A and A' and if they have not registered yet, it aborts. The rest of the simulation is similar to above.

Sim fully simulates the agent discovery protocol; the reason is that Sim can simulate the view of the corrupted agent (by simulating $\mathcal{F}_{\text{SCS}}$ and $\mathcal{F}_{\text{CA}}$ internally) and the output of the agent (which is the agent information and provider's signature) correctly using randomly generated values and the information it receives from the corrupted agent. We give the hybrid games for the in-distinguishability of the hybrid world from the ideal world in the full version of the paper.

F.4 Security Analysis of the A-session Protocol

In this section, we show that the A-sessionprotocol described in Section 4.4 (Figure 5) between two agents A_I and A_R UC realizes the A-sessionideal functionality $\mathcal{F}_{\text{Ases}}$.

Lemma F.4. *The A-session protocol (see Figure 20) of $\pi_{MAGIQ}^{\mathcal{F}_{\text{scs}},\mathcal{F}_{\text{CA}},\mathcal{G}_{\text{clk}},\mathcal{G}_h}$ (described in Section 4.4) UC realizes the A-session ideal functionality $\mathcal{F}_{\text{Ases}}$ against static adversaries.*

Proof. We consider three cases: (i) both A_I and A_R are honest, (ii) only A_I is corrupted, and (iii) only A_R is corrupted. We give the simulations below:

Case (i) is straightforward since $\mathcal{Z}$ only sees the output of honest agents. Assuming that the registration and agent discovery protocols have been simulated as described in previous subsections, and $\mathcal{F}_{\text{Ases}}$ simulates the request and responses internally, the view of $\mathcal{Z}$ is indistinguishable from the hybrid world. Note that Sim waits to receive ($Task, sid, A_I, attr_I$) from $\mathcal{F}_{\text{Ases}}$, and replies Ok. Then, whenever $\mathcal{F}_{\text{Ases}}$ aborts or terminates, it also aborts and terminates.

Below we only focus on the case where one of the agents is corrupted. Sim receives messages from the ideal functionality $\mathcal{F}_{\text{Ases}}$ and the corrupted agent. Based on these messages, it has to simulate the output of the honest parties and the full view of the corrupted parties. Note that Sim simulates $\mathcal{F}_{\text{scs}}$ and $\mathcal{F}_{\text{CA}}$ internally, and we assume it simulates the registration and agent discovery protocols as described in previous subsections.

Case (ii): A_I is corrupted. In this case, Sim^{A_I} has oracle access to the code of A_I, and it has to simulate the view of A_I and the output of the honest A_R. For that, Sim^{A_I} translates the messages it receives from the corrupted A_I for the ideal functionality $\mathcal{F}_{\text{Ases}}$and vice versa. It also simulates A_R's output.

Handshake. Sim^{A_I} waits until agent A_I initiates an A-session.

- Upon receiving the message ($m_0, \sigma_{ac}^{TA}, \sigma_{init}^{A_I}, \sigma_{ICP}^{U_I}$) from A_I, Sim^{A_I} parses $m_0 = (r_1, info_{A_I}, Q_{I,R}, \Delta_{I,R}, NextTk_0^{ICP})$ It sends

$(ChangePolicy, sid, APolicy)$ from the user U to $\mathcal{F}_{\text{Ases}}$ where $APolicy$ is $[(A_I, A_R), Q_{I,R}, \Delta_{I,R}$. Note that Sim^{A_I} simulates $\mathcal{F}_{\text{SCS}}$. It then verifies the validity of σ_{ac}^{TA}, $\sigma_{init}^{A_I}$, $\sigma_{ICP}^{U_I}$, and $info_{A_I}$, if they are valid it parses sid and gets $\mathcal{G}_h(tsk)$; Then, sends $Observe$ to $\mathcal{G}_h$ and retrieves tsk. It then sends $(Task, sid, tsk)$ from A_I to $\mathcal{F}_{\text{Ases}}$. Else, it sends $(Task, sid, \perp)$ to $\mathcal{F}_{\text{Ases}}$.

- Upon receiving the message $(Response, sid, A_R, attr_R, \ell = [Q_R^*, \Delta_R^*])$ from $\mathcal{F}_{\text{Ases}}$, store the message, and choose a symmetric key r_2, compute $k_{ses} = \mathcal{G}_h(r_1, r_2)$. Also compute the seed ρ_0 through $\mathcal{G}_h$ and compute the hash chain based on $task\text{-}msg\ Q_R$. Then, let $NextTok_0^{RCP} = (\rho_{Q_R^*}, \rho_{Q^*{}_R - 1})$, send $m_1 = r_2, Q_R^*, \Delta_R^*, NextTok_1^{RCP}, NextTok_1^{ICP}$. Also, compute a tag tag_1 using $\mathcal{G}_h$ on m_1 and k_{ses}, and send to A_I.

Message transmission.

- Upon receiving (m_i, tag_i) from A_I, Sim^{A_I} parses $m_i = (req_i, NextTok_i^{ICP}, NextTok_i^{RCP})$, then it checks the validity of tag_i on m_i and k_{ses} by querying $\mathcal{G}_h$. Also, program $\mathcal{G}_h$ by sending $(program, sid, NextTok_i^{ICP}, NextTok_{i-1}^{ICP})$. If $\mathcal{G}_h$ aborts or the checks do not pass, upon receiving $(Request, sid, A_I, attr_I, \ell)$ abort. Else, send ok to $\mathcal{F}_{\text{Ases}}$. Additionally, if $\mathcal{F}_{\text{Ases}}$aborts, Sim^{A_I} also aborts and terminates.
- Upon receiving a $(Response, sid, A_R, attr_R, \ell)$, Sim^{A_I} simulates the response to the request req_{i+1}.To simulate the responder, Sim^{A_I} executes the request internally using $ExeReq(A_R, req_{i+1})$ and gets the response res_{i+1}. Then, it generates $m_{i+1} = (res_{i+1}, NextTok_{i+1}^{ICP}, NextTok_{i+1}^{RCP})$, and computes the tag using $\mathcal{G}_h$ and k_{ses}. If $res_{i+1} \neq \perp$ send ok to $\mathcal{F}_{\text{Ases}}$. Else, aborts.

Note that if during the simulation $\mathcal{F}scs$ aborts, Sim^{A_I} simulates another instance of $\mathcal{F}_{\text{scs}}$. Also, whenever $\mathcal{F}_{\text{Ases}}$aborts, Sim^{A_I} aborts too.

Case (iii): A_R is corrupted. In this case, Sim^{A_R} has oracle access to the code of A_R, and it has to simulate the view of A_R and the output of the honest A_I. For that, Sim^{A_R} translates the messages it receives from the corrupted A_R for the ideal functionality $\mathcal{F}_{\text{Ases}}$and vice versa. It also simulates A_I internally.

Handshake. Sim^{A_R} waits until agent A_I initiates establishing the session with A_R.

- Upon receiving the message $(Request, sid, A_I, attr_I, \ell = [Q_I^*, \Delta_I^*])$ from $\mathcal{F}_{\text{Ases}}$, Sim^{A_R} simulates the establishment of a secure communication session $\mathcal{F}_{\text{scs}}$ between A_I and A_R. It also parses sid and extracts the task tsk by sending $Observe$ to $\mathcal{G}_h$, and stores tsk. It then simulates $(m_0, \sigma_{ac}^{TA}, \sigma_{init}^{A_I}, \sigma_{ICP}^{U_I})$ and send it to A_R, where $m_0 = (r_1, info_{A_I}, Q_I^*, \Delta_I^*, NextTk_0^{ICP})$ and r_1 is chosen uniformly at random. Also, $info_{A_I}$ is generated based on the information Sim^{A_R} has obtained during the previous protocols such as registration and agent discovery, or generated randomly from the identical distribution as the real values. Next, send ok to $\mathcal{F}_{\text{Ases}}$. If $\mathcal{F}_{\text{Ases}}$ aborts, Sim^{A_R} aborts and terminates.
- Upon receiving (m_1, tag_1) from A_R, Sim^{A_R} parses $m_1 = (r_2, Q_{R,I}, \Delta_{R,I}, NextTok_1^{RCP}, NextTok_1^{ICP})$, and verifies that tag_1 and $NextTok_1^{ICP}$ are valid. Also, parse $NextTok_1^{RCP} = [s_{Q_R^*}, s_{Q_R^*-1}]$ and program $\mathcal{G}_h$ to output $s_{Q_R^*}$ when queried by $s_{Q_R^*-1}$. If the checks pass and $\mathcal{G}_h$ does not abort, send ok to $\mathcal{F}_{\text{Ases}}$, else aborts and terminates.

Message transmission.

- Upon receiving a $(Request, sid, A_I, attr_I, \ell)$, Sim^{A_R} simulates the request req_i. To simulate the request, Sim^{A_I} executes the response internally using $ExeRes(A_R, tsk, req_i)$ and gets the response res_i. Then, it generates $m_i = (req_i, NextTok_i^{ICP}, NextTok_i^{RCP})$, and computes the tag using $\mathcal{G}_h$ and k_{ses}. If $req_i \neq \perp$, it sends ok to $\mathcal{F}_{\text{Ases}}$. Else, aborts.
- Upon receiving (m_{i+1}, tag_{i+1}) from A_R, Sim^{A_R} parses $m_{i+1} = (req_{i+1}, NextTok_{i+1}^{ICP}, NextTok_{i+1}^{RCP})$, then it checks the validity of tag_{i+1} on m_i and k_{ses} by querying $\mathcal{G}_h$. Also, program $\mathcal{G}_h$ by sending $(program, sid, NextTok_{i+1}^{RCP}, NextTok_i^{RCP})$. If $\mathcal{G}_h$ aborts or the checks do not pass, upon receiving $(Response, sid, A_R, attr_R, \ell)$ abort. Else, send ok to $\mathcal{F}_{\text{Ases}}$. Additionally, if $\mathcal{F}_{\text{Ases}}$aborts, Sim^{A_R} also aborts and terminates.

Note that if during the simulation $\mathcal{F}scs$ aborts, Sim^{A_R} simulates another instance of $\mathcal{F}_{\text{scs}}$. Also, whenever $\mathcal{F}_{\text{Ases}}$ aborts, Sim^{A_R} aborts too.

The above simulation completely simulates the hybrid A-session protocol since the simulator can extract the task, run the code of the honest agents on the task, and simulate the task messages correctly. Additionally, we assume that the signature scheme is EUF-CMA secure, so the corrupted agent cannot forge them, and the HMAC, hash function, and PRF are modeled by a random oracle, which means that their output are indistinguishable from random values. We give the hybrid games for the in-distinguishability of the hybrid world from the ideal world in the full version of the paper.

F.5 Security Analysis of the C-session Protocol

In this section, we show that the C-sessionprotocol described in Section 4.5 (Figure 5) between the *orchestrator* agent A_I^* and a set of A_R UC realizes the C-session ideal functionality $\mathcal{F}_{\text{Cses}}$.

LEMMA F.5. *The C-session protocol (see Figure 21) of $\pi_{MAGIQ}^{\mathcal{F}_{\text{scs}}, \mathcal{F}_{\text{CA}}, \mathcal{G}_{\text{clk}}, \mathcal{G}_h}$ (described in Section 4.4) UC realizes the C-session ideal functionality $\mathcal{F}_{\text{Cses}}$ against static adversaries assuming A-session π_{Ases} UC realizes $\mathcal{F}_{\text{Ases}}$.*

F.5.1 Proof. We consider three cases: (i) all agents are honest, (ii) only A_I^* is corrupted, and (iii) all A_R's are corrupted. We give the simulations below:

Case (i). This case is straightforward; when all agents are honest, Sim simulates the ideal functionalities $\mathcal{F}_{\text{Ases}}$ and $\mathcal{F}_{\text{A-auth}}$, and the output of honest agents correctly; it receives the task tsk from the dummy A_I^* and it executes it internally and gets the subtasks that should be simulated in each $\mathcal{F}_{\text{Ases}}$ (see case (iii)).

Case (ii). When A_I^* is corrupted, $\text{Sim}^{A_I^*}$ simulates the ideal functionalities $\mathcal{F}_{\text{Ases}}$ and $\mathcal{F}_{\text{A-auth}}$, and the view of the corrupted *orchestrator* (that is same as the simulated ideal functionalities) and output of honest agents correctly (see below):

- **Simulating each $\mathcal{F}_{\text{A-auth}}$.** $\text{Sim}^{A_I^*}$ waits to receive $(GetAuthorization, sid, A_I^*, attr_I, A_i, attr_i)$ for $A_i \in A_R$, and checks whether A_I^* and A_i are registered and their identities are in $CP_{A_I^*} \cap CP_{A_i}$. If so, it retrieves the counter and checks wether they have enough A-session budget. If so, it sends $(Allowed, sid, A_I^*, attr_I, A_i, attr_i)$ to A_I^* and stores it in L_{Auth}.
- **Simulating each $\mathcal{F}_{\text{Ases}}$.** For each A-session $\text{Sim}^{A_I^*}$ waits to receive $(AgentPolicy, sid, APolicy)$ for A_I^* and A_i, and stores

APolicy's. Upon receiving $(Task, sid, subtsk)$ from the corrupted A_I^*, it internally runs the task execution on *subtsk*, and returns the result and terminates when the task execution completes. Also, note that whenever A_I^* aborts, $\mathsf{Sim}^{A_I^*}$ aborts and terminates.

Case (iii). When A_i is corrupted, Sim^{A_i} simulates the ideal functionalities $\mathcal{F}_{\mathsf{Ases}}$ and $\mathcal{F}_{\mathsf{A\text{-}auth}}$, and the view of the corrupted A_i (that is same as the simulated $\mathcal{F}_{\mathsf{Ases}}$) and output of A_I^* correctly (see below):

- **Receiving the policy and task from A_I^*.** Sim^{A_i} waits to receive $(AgentPolicy, sid, APolicy)$ for A_I^* and $A_i \in A_R$, and stores *APolicy*'s. Upon receiving $(Task, sid, tsk)$ from dummy A_I^*, it internally runs the task execution on *tsk* and gets the schedule and the *subtask*'s for each responding agent A_i.
- **Simulating each $\mathcal{F}_{\mathsf{A\text{-}auth}}$.** Sim^{A_i} simulates the $\mathcal{F}_{\mathsf{A\text{-}auth}}$ for each A_i; it checks whether A_I^* and A_i are registered and their identities are in $CP_{A_I^*} \cap CP_{A_i}$. If so, it retrieves the counter and checks wether they have enough A-session budget. If the check passes, it sends $(Allowed, sid, A_I^*, attr_I, A_i, attr_i)$ to A_I^* and stores it in L_{Auth}.
- **Simulating each $\mathcal{F}_{\mathsf{Ases}}$.** Sim^{A_i} simulates $\mathcal{F}_{\mathsf{Ases}}$ for each *subtask* with A_i, and returns the result and terminates when the task execution completes. Whenever A_i aborts, Sim^{A_i} aborts and terminates.

The above simulation completely simulates the hybrid C-session protocol. We give the hybrid games for the in-distinguishability of the hybrid world from the ideal world in the full version of the paper.

Our analysis can be extended to the dynamic setting as well. We leave the complete proof for this setting to the full version of the paper.